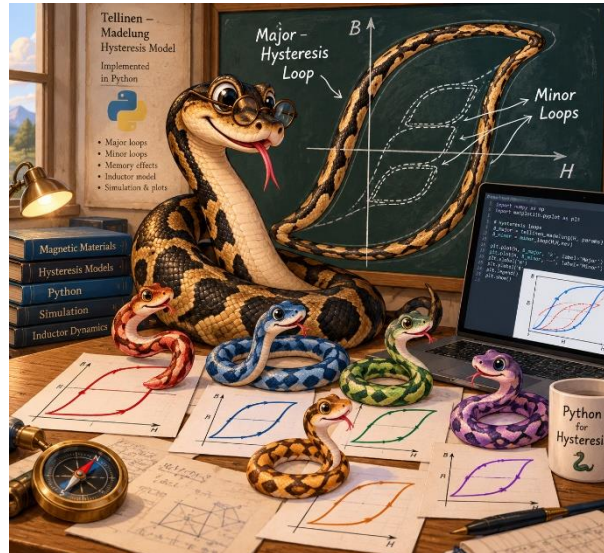


Tellinen–Madelung Magnetic Hysteresis Model and Its Implementation in Python



Contents

Introduction	3
Original Tellinen Magnetic Hysteresis Model	4
Incorporation of Madelung Magnetic Memory	6
Root Trajectory and Magnetic Memory Structure	7
First Reversal and First-Order Return Curve	7
Second and Subsequent Reversals	7
Mapping the Tellinen Branch to the Madelung Return Point	8
Wiping-Out and Restoration of the Parent Branch	8
Numerical Evaluation of the Integrals	9
Trapezoidal Method with Finite Differences	9
More Accurate Method Based on Quadratic Lagrange Interpolation	10
Algorithmic Implementation of the Tellinen–Madelung Model	11
Structure of the Tellinen–Madelung Python Project	13
main.py — Main Control Module	14
BH.py — Core of the Magnetic Model	16
Parameters.py	20
Parameters.txt	21

Hscan.py.....	21
Major_Loop.csv	22
Comparison of Calculations Using the Tellinen–Madelung and Chan–Engelhardt Models	22
Magnetic-Core Inductor Model.....	24
Algorithmic Implementation of the Inductor Calculation	27
Structure of the Python Project for Inductor Calculation.....	30
main.py — Main Inductor Calculation Module.....	31
Bracketing for the Bisection Method	31
Solution of the Nonlinear Equation.....	32
Committing the Accepted State	32
Parameters.py and Parameters.txt.....	33
V.py.....	33
Output Files.....	34
Comparison with LTspice Inductor Simulations	34
References	42

Introduction

Magnetic hysteresis differs fundamentally from an ordinary nonlinear dependence $B(H)$ in that the magnetic flux density is determined not only by the current value of the magnetic field strength, but also by the history of its variation. At the same H the material can be in different magnetic states depending on the trajectory by which it arrived at the point under consideration. Therefore, a correct model must describe not only the shape of the major loop, but also the mechanism by which internal branches are formed under successive field reversals. The [review](#) by Mörée and Leijon [1] emphasizes that the way magnetic history is stored is one of the main features distinguishing modern hysteresis models.

In the proposed model, the shape of the local branches is constructed on the basis of the Tellinen model. Its main idea is that the differential permeability of an internal trajectory is determined by the slope of the corresponding branch of the major loop multiplied by a coefficient characterizing the position of the current point between the upper and lower boundaries of the loop. When the field increases, the derivative of the lower branch of the major loop is used, whereas when the field decreases, the derivative of the upper branch is used. As a result, the internal trajectories are directly related to the geometry of the actual major loop and are naturally confined to the region between its two branches. Unlike some simpler scaling models, this construction substantially reduces the risk of nonphysical branches extending beyond the major loop. However, the standard Tellinen model has only local memory: subsequent motion is determined by the current values H , B and by the direction of field variation. This is sufficient to construct individual return curves, but not to reproduce rigorously a complex sequence of nested minor loops. After several reversals, the model itself does not retain information about which of the earlier return points the current branch should reach.

To overcome this limitation, the proposed model uses the Madelung magnetic-memory principle illustrated in Fig. 1. The magnetization history is represented as a sequence of reversal points $P_1, P_2, \dots, P_n$ at which the sign of the magnetic-field variation changes. A new branch starting from the latest point P_n must return to the preceding point P_{n-1} . If this return occurs and the motion continues further, the corresponding inner loop is removed from memory and the trajectory restores an earlier branch. These properties are known as return-point memory and wiping-out memory. In the Madelung model, the sequence of reversal points itself provides a compact representation of the magnetic history.

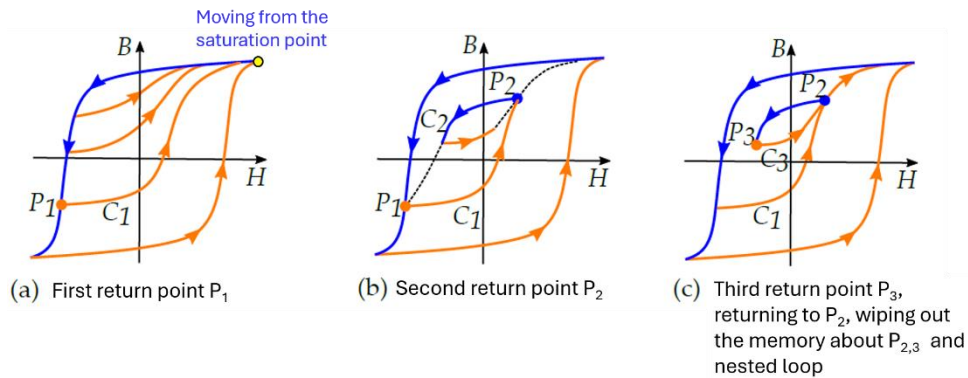


Fig. 1. Rules of Madelung magnetic memory.

Thus, the two parts of the model perform different functions. Tellinen determines the shape of the current branch using the geometry of the major loop and the position of the point inside the hysteresis region, whereas Madelung determines the topology of the magnetic memory — that is, from which point a branch starts, to which preceding point it must return, and when a nested loop must be erased from the history. Consequently, the magnetic dependence is more correctly written not as a single-valued function $B = B(H)$, but as $B = B(H, \mathcal{R})$, where $\mathcal{R} = \{P_1, P_2, \dots, P_n\}$ represents the stored history of reversal points. This approach is particularly convenient for software implementation. At any instant it is necessary to store only the current accepted magnetic state, the direction of field variation, and the stack of reversal points. At the next reversal, a new point is added to the stack. When a nested loop closes, the corresponding pair of points is removed. The model therefore naturally permits first-, second-, third-, and higher-order loops without introducing a separate set of special rules for each case.

It is important to emphasize that the Tellinen + Madelung combination considered below is not a ready-made standard model in the form in which it appears in the literature. In review [1], Tellinen is treated as an independent differential scaling model, whereas the Madelung rules are treated as a separate principle of history dependence. Here they are combined into a single computational scheme: the local branch shape is inherited from Tellinen, while the exact memory and return structure is inherited from Madelung. This combination preserves the simplicity of the Tellinen differential model while providing multilevel memory for nested minor loops.

Original Tellinen Magnetic Hysteresis Model

We denote the upper and lower branches of the major loop as:

$$C_u(H) \tag{1}$$

$$C_l(H) \tag{2}$$

The width of the hysteresis region at a given H is:

$$D(H) = C_u(H) - C_l(H) \tag{3}$$

For any physically admissible point inside the major loop, the following condition must hold: $C_l(H) \leq B \leq C_u(H)$.

In the original Tellinen model, the internal branches are specified directly through the differential permeability. For increasing H :

$$\frac{dB}{dH} = k_{\uparrow}(H, B) \frac{dC_l}{dH} \tag{4}$$

where

$$k_{\uparrow}(H, B) = \frac{C_u(H) - B}{C_u(H) - C_l(H)} \tag{5}$$

For decreasing H :

$$\frac{dB}{dH} = k_{\downarrow}(H, B) \frac{dC_u}{dH} \quad (6)$$

where

$$k_{\downarrow}(H, B) = \frac{B - C_l(H)}{C_u(H) - C_l(H)} \quad (7)$$

This is the basic Tellinen construction. Its attractive feature is that the scaling coefficient is determined by the position of the current point between the two branches of the major loop, rather than by the distance to saturation. Therefore, the model more naturally keeps return curves inside the major-loop region than some simpler scaling models, for example the original Chan model. The physical meaning is very clear. If, while the field is increasing, we are already on the lower major curve $B = C_l(H)$, then $k_{\uparrow} = 1$ and $\frac{dB}{dH} = \frac{dC_l}{dH}$. That is, the trajectory remains on the major loop. If, for increasing H , the point lies close to the upper branch, $B \rightarrow C_u(H)$, then $k_{\uparrow} \rightarrow 0$, and the slope of the new return curve becomes small. For a decreasing field, the situation is mirrored.

For solving the differential equations, it is convenient to introduce a new function:

$$y(H, B) = \frac{B - C_l(H)}{C_u(H) - C_l(H)} \quad (8)$$

The relation $0 \leq y \leq 1$. Here $y = 0$ corresponds to the lower major curve, whereas $y = 1$ corresponds to the upper one. The magnetic flux density is recovered from y as:

$$B(H) = C_l(H) + yD(H) \quad (9)$$

In this notation, the Tellinen coefficients become particularly simple:

$$k_{\uparrow} = 1 - y \quad (10)$$

$$k_{\downarrow} = y \quad (11)$$

The original Tellinen differential equation can now be rewritten using y . From (9) and (3), we have:

$$\frac{dB}{dH} = \frac{dC_l}{dH} + D \frac{dy}{dH} + y \left(\frac{dC_u}{dH} - \frac{dC_l}{dH} \right) \quad (12)$$

Then, for an increasing loop ($H \uparrow$), we obtain:

$$\frac{dB}{dH} = \frac{dC_l}{dH} + D \frac{dy}{dH} + y \left(\frac{dC_u}{dH} - \frac{dC_l}{dH} \right) = (1 - y) \frac{dC_l}{dH} \quad (13)$$

After simplification, we obtain:

$$(H \uparrow) \quad \frac{dy}{dH} = - \frac{y}{D(H)} \frac{dC_u}{dH} \quad (14)$$

For a decreasing branch ($H \downarrow$), we obtain:

$$\frac{dB}{dH} = \frac{dC_l}{dH} + D \frac{dy}{dH} + y \left(\frac{dC_u}{dH} - \frac{dC_l}{dH} \right) = y \frac{dC_u}{dH} \quad (15)$$

After simplification, we obtain:

$$(H \downarrow) \frac{dy}{dH} = - \frac{(1-y)}{D(H)} \frac{dC_l}{dH} \quad (16)$$

These two formulas are extremely convenient because they allow the Tellinen branches to be obtained not by sequential Euler integration, but directly as a function of H .

Let the return loop start at point $P_a = (H_a, B_a)$. At this point:

$$y_a = \frac{B_a - C_l(H_a)}{D(H_a)} \quad (17)$$

For the increasing branch, from (14) we have:

$$\frac{dy}{y} = - \frac{1}{D(H)} \frac{dC_u}{dH} dH \quad (18)$$

Integrating both sides of (18) with the initial condition (17), we obtain:

$$y_T(H_{\uparrow}) = y_a \exp \left[- \int_{H_a}^H \frac{1}{D(H)} \frac{dC_u}{dH} dH \right], \quad H > H_a \quad (19)$$

where the index T denotes the original Tellinen branch. For the decreasing branch, it is convenient to introduce a new function:

$$z = 1 - y \quad (20)$$

Then, from (16), we obtain:

$$\frac{dz}{z} = \frac{1}{D} \frac{dC_l}{dH} dH, \quad (21)$$

Integrating both sides of (21), we obtain:

$$z_T(H_{\downarrow}) = z_a \exp \left[\int_{H_a}^H \frac{1}{D(H)} \frac{dC_l}{dH} dH \right], \quad H < H_a \quad (22)$$

where $z_a = 1 - y_a$. Therefore:

$$y_T(H_{\downarrow}) = 1 - (1 - y_a) \exp \left[\int_{H_a}^H \frac{1}{D(H)} \frac{dC_l}{dH} dH \right], \quad H < H_a \quad (23)$$

Incorporation of Madelung Magnetic Memory

The standard Tellinen model belongs to models with local memory: subsequent motion is determined by the current values H and B and by the direction of field variation, but the sequence of previous reversal points is not stored explicitly. Therefore, after several successive reversals, the conventional Tellinen model itself contains no strict rule specifying which of the previously visited points the current branch should return to. To introduce multilevel magnetic memory, we use the Madelung rules already mentioned in the Introduction. Their meaning is as follows. The first return curve is determined by the point from which it originated. If a new reversal occurs on this curve, the next

branch must return to the preceding reversal point. After a nested loop closes, the corresponding history is erased and the motion continues along the older parent branch as if the inner loop had never existed. These properties correspond to return-point memory and the wiping-out rule.

Root Trajectory and Magnetic Memory Structure

In the computational implementation, the root trajectory and the stack of reversal points must be distinguished. The root trajectory $B_{root}(H)$ is the branch to which the system must return after all nested loops have been completely wiped out. The model considered here has three possible root states: the initial magnetization curve $B_{init}(H)$, the upper branch of the major loop $C_u(H)$, and the lower branch of the major loop $C_l(H)$. It is convenient to store them in the variable `root_branch` with the values "initial", "upper_major", and "lower_major": $B_{root}(H) = B_{init}(H)$, $C_u(H)$, or $C_l(H)$, depending on `root_branch`.

The `reversal_stack` contains only real points at which the direction of variation of H actually changed. Virtual or auxiliary points are not placed in the stack. If the real reversal points are denoted by $P_i = (H_i, B_i)$, the magnetic state can be represented by the set H_{last} , B_{last} , `previous_direction`, `root_branch`, and `reversal_stack`. An empty `reversal_stack` means that there is no active nested memory. In this case, the system moves not along a new Tellinen branch but directly along the stored root trajectory. Consequently, after complete wiping-out, the model restores exactly the branch on which it was located before the nested structure arose.

First Reversal and First-Order Return Curve

The first reversal requires separate consideration because the memory does not yet contain a preceding real reversal point that could serve as the Madelung target point. Regardless of whether the root trajectory is $B_{init}(H)$, $C_u(H)$, or $C_l(H)$, after the first reversal at $P_1 = (H_1, B_1)$ the `reversal_stack` contains only P_1 . Therefore, the original Tellinen law is applied to the first-order return curve without a Madelung endpoint constraint. In other words, the first return curve is an ordinary Tellinen FORC starting at P_1 .

If the next reversal occurs on the FORC, a second real point P_2 appears, and from that moment a uniquely defined target point P_1 exists. If no further reversals occur, the original Tellinen FORC approaches the corresponding major branch as the field changes: for increasing H , it approaches $C_l(H)$, and for decreasing H , it approaches $C_u(H)$. When this branch is reached within the specified numerical tolerance, point P_1 is removed, the stack is cleared, and the corresponding major branch becomes the new `root_branch`.

Second and Subsequent Reversals

After each subsequent change of direction, the accepted point $P_n = (H_n, B_n)$ is added to `reversal_stack`. Starting with the second real reversal point, the current branch has uniquely defined start and target points: `start=stack[-1]`, `target=stack[-2]`. In other words, the new branch starting from the latest point P_n must return to the immediately preceding point P_{n-1} . This rule forms the nested structure of Madelung memory: `Pstart = stack[-1]`, `Ptarget = stack[-2]`, if `len(reversal_stack) ≥ 2`.

Mapping the Tellinen Branch to the Madelung Return Point

An ordinary Tellinen branch launched from $P_a = (H_a, B_a)$ is not, in general, required to pass exactly through the stored Madelung target point $P_b = (H_b, B_b)$. Therefore, organizing the stack alone is insufficient: the characteristic shape of the Tellinen branch must be preserved while ensuring exact satisfaction of the return condition. For this purpose, normalization is performed directly with respect to the magnetic flux density B . Let $B_T(H)$ be the ordinary Tellinen branch calculated from P_a using (19) or (23). Denote by B_{Tb} the value of this uncorrected branch at $H = H_b$:

$$B_{Tb} = B_T(H_b) \quad (24)$$

To hit the endpoint exactly, introduce a scaling coefficient with respect to B :

$$A_B = \frac{B_b - B_a}{B_{Tb} - B_a} \quad (25)$$

The Madelung-constrained branch is then defined as:

$$B_{TM}(H) = B_a + A_B[B_T(H) - B_a] \quad (26)$$

Equations (24)–(26) form the modification of the original Tellinen model proposed here. Tellinen determines the shape of the variation along the branch, whereas Madelung specifies the exact endpoint. If the uncorrected Tellinen branch already reaches the target point without correction, then $B_{Tb} = B_b$ and $A_B = 1$, so the corrected branch coincides with the original one. An important property of normalization directly with respect to B is that $\frac{dB_{TM}}{dH} = A_B \frac{dB_T}{dH}$. Therefore, for $A_B > 0$, the sign of the differential permeability of the original Tellinen branch is preserved. In the algorithm, a non-positive value of A_B is treated as inadmissible.

Wiping-Out and Restoration of the Parent Branch

When the current branch reaches P_{target} , the calculation must first arrive exactly at the stored point (H_{target}, B_{target}) and only then modify the memory. After the corresponding nested loop closes, the last two points — P_{start} and P_{target} — are removed from `reversal_stack`. The subsequent action is determined not by the fact of removal itself, but by the number of real reversal points remaining. If two or more points remain in the stack after removal, the root trajectory is not yet restored. The older parent branch defined by the new pair `stack[-1]` and `stack[-2]` becomes active. If one point remains, the first-order return curve that previously started from that point is restored. Only when `reversal_stack` becomes completely empty is the entire nested history considered exhausted and the subsequent motion returns to `root_branch`: $\text{len}(\text{stack}) \geq 2 \rightarrow$ higher-order parent branch; $\text{len}(\text{stack}) = 1 \rightarrow$ FORC; $\text{len}(\text{stack}) = 0 \rightarrow$ root branch. For example, for `reversal_stack` = $[P_1, P_2, P_3, P_4]$, branch $P_4 \rightarrow P_3$ is active. After P_3 is reached, the closed inner loop is wiped out and the stack becomes $[P_1, P_2]$. The system continues along the restored parent branch $P_2 \rightarrow P_1$. Only after P_1 is reached and the pair $[P_1, P_2]$ is removed does the stack become empty, after which the root trajectory is restored. Thus, the presence of even one still-active reversal point means that a complete return to `root_branch` has not yet occurred.

The rule for restoring a first-order return curve is the same for all three types of `root_branch`. If a single point P_1 remains in the stack after wiping-out, the system returns to the original Tellinen FORC

that previously started from this point. If no further reversals occur, this FORC continues to the corresponding major branch, after which P_1 is removed from memory, the stack is cleared, and the reached major branch becomes the new root_branch.

Numerical Evaluation of the Integrals

In (19) and (23), we need to evaluate integrals of the following form:

$$I = \int_a^b f(x) \frac{dg(x)}{dx} dx \quad (27)$$

where $f(x)$ and $g(x)$ are sufficiently smooth functions. To avoid ambiguity in the notation, let N denote the number of intervals. Then $N + 1$ nodes are used: $x_i = a + i\Delta x, i = 0, 1, \dots, N$, with step $\Delta x = \frac{b-a}{N}$. Denote $f_i = f(x_i)$ and $g_i = g(x_i)$.

Trapezoidal Method with Finite Differences

Apply the standard trapezoidal rule to the integrand $F(x) = f(x)g'(x)$:

$$I \approx \Delta x \left[\frac{f_0 g'_0}{2} + \sum_{i=1}^{N-1} f_i g'_i + \frac{f_N g'_N}{2} \right] \quad (28)$$

At the interior points, the derivative $g'(x)$ is evaluated using the central two-sided difference $g'_i \approx \frac{g_{i+1} - g_{i-1}}{2\Delta x}, i = 1, \dots, N-1$. At the left boundary a one-sided difference $g'_0 \approx \frac{g_1 - g_0}{\Delta x}$, is used, and at the right boundary — $g'_N \approx \frac{g_N - g_{N-1}}{\Delta x}$.

After substituting these expressions into (28), the step Δx cancels, giving:

$$I \approx \frac{1}{2} f_0 (g_1 - g_0) + \frac{1}{2} \sum_{i=1}^{N-1} f_i (g_{i+1} - g_{i-1}) + \frac{1}{2} f_N (g_N - g_{N-1}) \quad (29)$$

This expression can be simplified substantially. Note that $g_{i+1} - g_{i-1} = (g_i - g_{i-1}) + (g_{i+1} - g_i)$. After grouping the terms corresponding to each individual increment $\Delta g_i = g_{i+1} - g_i$, we obtain:

$$I \approx \sum_{i=0}^{N-1} \frac{f_i + f_{i+1}}{2} (g_{i+1} - g_i) \quad (30)$$

For sufficiently smooth functions, the local error on one interval is of order $O(\Delta x^3)$, and after summation over $N \sim \frac{1}{\Delta x}$ intervals the global error is $I - I_h = O(\Delta x^2)$. Thus, this is a second-order method.

More Accurate Method Based on Quadratic Lagrange Interpolation

The accuracy of the integral evaluation can be increased by using quadratic Lagrange polynomials over each pair of adjacent intervals instead of linear interpolation. Consider three consecutive nodes x_i, x_{i+1}, x_{i+2} , with $x_{i+1} - x_i = x_{i+2} - x_{i+1} = \Delta x$. Introduce the dimensionless coordinate $t = \frac{x - x_i}{\Delta x}$. Then $0 \leq t \leq 2$, and the three nodes correspond to $t = 0, 1, 2$. The Lagrange polynomials have the form

$$L_0(t) = \frac{(t-1)(t-2)}{2} \quad (31)$$

$$L_1(t) = -t(t-2) = t(2-t), \quad (32)$$

$$L_2(t) = \frac{t(t-1)}{2} \quad (33)$$

Approximate both functions by quadratic polynomials:

$$f(x) \approx f_i L_0(t) + f_{i+1} L_1(t) + f_{i+2} L_2(t) \quad (34)$$

$$g(x) \approx g_i L_0(t) + g_{i+1} L_1(t) + g_{i+2} L_2(t) \quad (35)$$

Over the double interval under consideration, the following partial integral must be evaluated:

$$I_i = \int_{x_i}^{x_{i+2}} f(x) g'(x) dx \quad (36)$$

Since $x = x_i + t\Delta x$, $dx = \Delta x dt$, and $\frac{dg}{dx} = \frac{1}{\Delta x} \frac{dg}{dt}$, the factors Δx cancel:

$$I_i = \int_0^2 f(t) \frac{dg(t)}{dt} dt \quad (37)$$

Substituting (34) and (35) into (36), we obtain:

$$I_i \approx \sum_{j=0}^2 \sum_{k=0}^2 f_{i+j} g_{i+k} \int_0^2 L_j(t) L'_k(t) dt \quad (38)$$

Denote $A_{jk} = \int_0^2 L_j(t) L'_k(t) dt$. Direct integration gives the matrix:

$$\begin{pmatrix} -\frac{1}{2} & \frac{2}{3} & -\frac{1}{6} \\ -\frac{2}{3} & 0 & \frac{2}{3} \\ \frac{1}{6} & -\frac{2}{3} & \frac{1}{2} \end{pmatrix} \quad (39)$$

We obtain:

$$I_i \approx f_i \left(-\frac{1}{2}g_i + \frac{2}{3}g_{i+1} - \frac{1}{6}g_{i+2} \right) + f_{i+1} \left(-\frac{2}{3}g_i + \frac{2}{3}g_{i+2} \right) + f_{i+2} \left(\frac{1}{6}g_i - \frac{2}{3}g_{i+1} + \frac{1}{2}g_{i+2} \right) \quad (40)$$

This is the basic three-point formula of the method. If the number of intervals N is even, the entire range is divided into pairs of intervals: $[x_0, x_2], [x_2, x_4], \dots, [x_{N-2}, x_N]$. Then:

$$I \approx \sum_{i=0}^{N-2} I_i \quad (41)$$

where each I_i is calculated from (40). If N is odd, a linear approximation can be used for the last remaining interval. In practice, it is even simpler to choose an even N .

For smooth $f(x)$ and $g(x)$ over a double interval of length $2\Delta x$, a Taylor-series expansion shows that the local error begins with the fifth power of the step: $I_i^{(L)} - I_i = O(\Delta x^5)$. After summation over approximately $\frac{N}{2} \sim \frac{1}{\Delta x}$ double intervals, the global error is $I - I_h = O(\Delta x^4)$. Therefore, the quadratic Lagrange scheme has fourth-order accuracy, whereas the linear scheme is second order.

Algorithmic Implementation of the Tellinen–Madelung Model

Below is the complete algorithm for one accepted step. It is formulated specifically so that the same state object can later be used safely in the nonlinear inductor solver, where trial calculations $B(H)$ must be performed without modifying the accepted magnetic memory.

1. Initialization. The parameters `root_branch` and H_{start} are specified in `Parameters.txt`. Three starting options are allowed: "initial", "upper_major", and "lower_major". The value B_{start} is not specified separately but is calculated automatically from the selected root trajectory: $B_{start} = B_{root}(H_{start})$. The field H_{start} must lie within the common range of the upper and lower branches $H \in [H_{min}, H_{max}]$; this is checked when the parameters are read. If a separate initial curve is provided and `root_branch`="initial", its range is checked additionally. For "upper_major", `previous_direction`="dec" is set; for "lower_major", `previous_direction`="inc" is set; for the initial curve the direction is determined by the sign of H_{start} , and for $H_{start} = 0$ `previous_direction`=None is adopted. The `Hscan` functions are retained unchanged; the field history generated by them is shifted so that its first point coincides exactly with H_{start} .

2. Check for field change. For a new H , $\Delta H = H - H_{last}$ is calculated. If $|\Delta H| < H_{tol}$, the field is considered unchanged: `Blast` is returned and the entire state is preserved without modification.

3. Direction determination. For $\Delta H > 0$, `direction`="inc" is adopted; for $\Delta H < 0$, `direction`="dec".

4. Registration of a new reversal point. If `previous_direction`=None, the new direction is only stored. If `direction` coincides with `previous_direction`, no new memory point is created. If the directions differ, the accepted point $P = (H_{last}, B_{last})$ is added to `reversal_stack`, after which `previous_direction` is replaced by the new direction.

5. Classification of the current branch. The subsequent law is determined by the length of `reversal_stack`. For `len(stack)`=0, motion follows `root_branch`; for `len(stack)`=1, a first-order return curve is active; for `len(stack)`≥2, a higher-order Madelung-constrained branch is active.

6. Empty stack. If reversal_stack=[], $B = B_{root}(H)$ is calculated. For root_branch="initial" this is $B_{init}(H)$, for "upper_major" — $C_u(H)$, and for "lower_major" — $C_l(H)$. No new Tellinen branch is created in this case.

7. One point in the stack. Let stack=[P_1]. There is not yet a preceding real reversal point, so a Madelung endpoint constraint cannot be applied. Regardless of the root_branch type, an ordinary first-order Tellinen reversal curve is constructed from P_1 without a Madelung endpoint correction. If the next reversal occurs, P_2 is added to stack and a specific target point P_1 appears.

8. Completion of the first-order return curve. If no further reversals occur, the original Tellinen FORC approaches the corresponding major branch. For direction="inc" this is $C_l(H)$, and for direction="dec" it is $C_u(H)$. If the difference between the FORC and this major branch becomes no greater than B_{tol} , point P_1 is removed, stack is cleared, and the reached major branch becomes the new root_branch.

9. Two or more points. For len(stack)≥2, start=stack[-1] and target=stack[-2] are assigned. The current branch is calculated by tellinen_madelung_branch(H,start,target), which uses equations (24)–(26).

10. Checking whether the target point has been reached. For an increasing branch, the target is considered reached or passed when $H \geq H_{target} - H_{tol}$, and for a decreasing branch when $H \leq H_{target} + H_{tol}$. Until this condition is satisfied, the stack is not changed.

11. Motion toward the target. If the target has not yet been reached, the calculated H and B become the current result of the step, while reversal_stack remains unchanged.

12. Closing a nested loop until the memory is fully exhausted. If the target is reached, the trajectory is first set exactly to (H_{target}, B_{target}) . Then P_{start} and P_{target} are removed from the stack. If len(stack)≥2 after this, the system immediately restores the older parent branch defined by the new pair stack[-1], stack[-2]. If len(stack)=1, the corresponding first-order parent branch is restored. In both cases root_branch is not yet activated: as long as at least one real reversal point remains in the stack, the nested magnetic history has not yet been exhausted.

13. Complete exhaustion of reversal points. If wiping-out results in reversal_stack=[], the subsequent trajectory immediately returns to the stored root_branch. For the original demagnetized state this is $B_{init}(H)$; for loops generated on the major loop, it is the corresponding major branch. This transition implements the complete wiping-out rule.

14. Passing several return points in one step. The final specified H may lie beyond one or more stored return points. Therefore, crossing must be processed in a while loop: after each inner loop is closed, the parent branch is restored and it is checked again whether the next target point has also been passed. A single outer calculation step can therefore close several nested loops at once.

15. Committing the accepted state. After all crossings have been processed, the final H and B are stored as H_{last} and B_{last} , together with the updated previous_direction, root_branch, and reversal_stack. Only this state is used as the initial state for the next accepted step.

For the future nonlinear inductor calculation, it is important that the branch-evaluation functions do not modify the passed state in-place. Each trial value H_{trial} must be calculated from a copy of the accepted state; changes to the stack and root_branch are committed only after the corresponding point is accepted by the solver.

The experimental major-loop curves $C_u(H)$ and $C_l(H)$ will be read from the file “Major_Loop.csv”, where they will be stored as two pairs of columns: $(H, C_u(H))$, $(H, C_l(H))$, where $H \in [-H_{max}, H_{max}]$. A third pair of columns, if present, will contain $(H, B_{init}(H))$, where $H \in [-H_{max}, H_{max}]$. If this pair of columns is absent, the initial magnetization curve will not be read but will be calculated as $B_{init}(H) = \frac{C_u(H) + C_l(H)}{2}$. Despite the common range $H \in [-H_{max}, H_{max}]$, the number of points in each pair of columns may differ, so they must be read separately. After reading the discrete values $C_u(H)$ and $C_l(H)$, shape-preserving piecewise-cubic PCHIP interpolation functions will be created, allowing values to be evaluated at any point H . The same is done for the discrete values $B_{init}(H)$ if they are present in the file. Otherwise, the continuous initial-magnetization function is calculated from the interpolated images $C_u(H)$ and $C_l(H)$ as their average. For test calculations, the column pairs $(H, C_u(H))$, $(H, C_l(H))$, and $(H, B_{init}(H))$ can be generated from the Chan model.[2]

Structure of the Tellinen–Madelung Python Project

The Python implementation of the model follows the same general scheme as the previously developed [Chan–Engelhardt](#) project.[3] This preserves the familiar program organization and, in the future, makes it possible to use almost the same interface when incorporating the hysteresis model into the nonlinear inductor calculation. The main difference is that the analytical Chan major-loop functions are replaced by tabulated dependences read from Major_Loop.csv, while the magnetic-memory state is now specified by the root branch and the stack of reversal points.

The structure of the project stored in the [GitHub](#) repository,[4] is as follows:

```
Tellinen-Madelung-Model-BHloops/
|
|— main.py
|— BH.py
|— Hscan.py
|— Parameters.py
|— Parameters.txt
|— Major_Loop.csv
|— BHloop.csv
```

Major_Loop.csv is the input file containing the magnetic characteristics of the material, while BHloop.csv is created by the program and contains the calculated hysteresis trajectory. The overall program structure can ultimately be represented by the following sequence:

Parameters.txt → *Parameters.py*

specify the initial state and numerical parameters;

Major_Loop.csv → *BH.py*

specifies the properties of the magnetic material;

Hscan.py → *H(t)*

specifies the excitation history; then

$$\boxed{\text{main.py} \rightarrow BH_{\text{increasing/decreasing}} \rightarrow \text{_advance}}$$

determines the sequence of magnetic states;

and inside BH.py

$$\boxed{\text{Tellinen + Madelung stack} \rightarrow B(H)}$$

forms the current hysteresis branch.

Final result:

$$\boxed{H(t) \rightarrow B(t)}$$

is saved as the trajectory

$$\boxed{BHloop.csv.}$$

Compared with the [Chan–Engelhardt](#) project, the division of responsibilities is particularly clear here: Major_Loop.csv defines the material properties, Tellinen defines the local branch shape, the Madelung stack provides magnetic memory, and main.py only advances the system sequentially along the prescribed history $H(t)$. This same separation will later make it possible to incorporate the model into the nonlinear inductor calculation with virtually no changes to the magnetic core module.

main.py — Main Control Module

The main.py file acts as the top-level module of the program. It does not itself contain the Tellinen mathematical equations or the Madelung algorithm, but organizes the sequential calculation of the trajectory $B(H)$. The initial magnetic state and numerical parameters are read from Parameters.py: H_{start} , root_branch , H_{tol} .

The variable:

```
root_branch
```

can take three values:

```
initial
upper_major
lower_major
```

and determines the root trajectory on which the material is located before the calculation begins. The initial value of the magnetic flux density is not specified separately by the user, but is calculated directly from the selected curve: $B_{\text{start}} = B_{\text{root}}(H_{\text{start}})$. For this purpose, main.py calls the function `root_branch_value(Hstart, root_branch)` from BH.py. Thus, if, for example, the following is specified:

```
root_branch=upper_major
Hstart=200
```

the program automatically sets $B_{\text{start}} = C_u(200)$. This excludes the possibility of specifying an initial point that does not belong to the selected magnetic branch.

After determining the initial state, `main.py` creates the time grid $t_0, t_1, \dots, t_N$ and calls one of the functions from `Hscan.py`, for example:

```
Hscan1(t)
```

or

```
Hscan2(t)
```

The `Hscan.py` functions themselves are retained unchanged from the [Chan–Engelhardt](#) project. To allow the same scanning function to be used for different initial field values, its result is treated as a field increment relative to the initial value: $H(t) = H_{\text{start}} + H_{\text{scan}}(t) - H_{\text{scan}}(t_0)$. Therefore, regardless of the particular `Hscan` form: $H(t_0) = H_{\text{start}}$. This approach allows the initial magnetic state to be changed only through `Parameters.txt`, without rewriting the test excitation functions. After the `Hvalues` array is formed, the program checks that the entire required trajectory lies within the field range available in `Major_Loop.csv`: $H_{\text{min}} \leq H(t) \leq H_{\text{max}}$. If this range is exceeded, the calculation stops with an error message.

Unlike Chan–Engelhardt, where a set of parameters $B_a, B_{a,\text{turn}}, H_0, \text{scale}, \text{offset}$, had to be stored, the state in the Tellinen–Madelung model has a considerably more direct physical meaning. At each instant, the following are stored: $H_{\text{last}}, B_{\text{last}}, \text{previous_direction}, \text{root_branch}, \text{reversal_stack}$. Here H_{last} and B_{last} are the last accepted point of the trajectory.

`previous_direction` can take the values:

```
"inc"
"dec"
None
```

and indicates the direction of the previous change in H .

`root_branch` defines the root trajectory that must be restored after the nested magnetic memory has been completely wiped out.

`reversal_stack` is a list of real reversal points: $\text{reversal_stack} = [P_1, P_2, \dots, P_n]$, where $P_i = (H_i, B_i)$. This list implements the Madelung memory.

When starting on the major loop, the direction of motion is already physically associated with the selected branch. For the upper major branch, the following is adopted:

```
previous_direction = "dec"
```

because the upper branch of the major loop corresponds to motion of the field in the decreasing direction. For the lower branch:

```
previous_direction = "inc"
```

If the calculation starts on the initial curve $B_{\text{init}}(H)$, the direction is determined by the sign of H_{start} . For $H_{\text{start}} > 0$, the system is assumed to have arrived from the demagnetized state with increasing field, so

previous_direction="inc". For $H_{\text{start}} < 0$, previous_direction="dec" is used. Only when starting from $H_{\text{start}} = 0, B_{\text{start}} = 0$ is the direction still undefined:

```
previous_direction=None
```

For each new value H_i , the program compares it with the previous H_{last} . If $|H_i - H_{\text{last}}| \leq H_{\text{tol}}$, the magnetic field is considered unchanged and $B_i = B_{\text{last}}$. The memory state is not changed. If $H_i > H_{\text{last}}$, the following is called:

```
BH_increasing(...)
```

from BH.py. If $H_i < H_{\text{last}}$, the following is called:

```
BH_decreasing(...)
```

Both functions receive the current accepted state: $H_{\text{last}}, B_{\text{last}}, \text{root_branch}, \text{reversal_stack}$ and return $B_i, \text{root_branch}^{\text{new}}, \text{reversal_stack}^{\text{new}}$. After this, $H_{\text{last}} \leftarrow H_i, B_{\text{last}} \leftarrow B_i$, and the calculation proceeds to the next step.

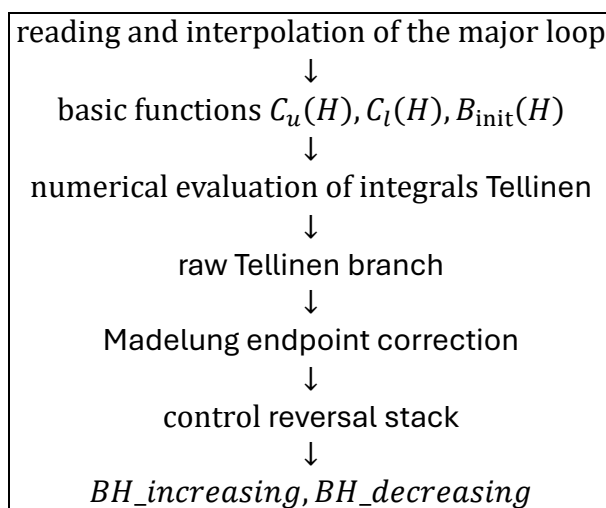
After the entire array has been processed, the following is formed

$$BH_{\text{loop}} = \begin{pmatrix} H_0 & B_0 \\ H_1 & B_1 \\ \vdots & \vdots \\ H_N & B_N \end{pmatrix},$$

the graph $B(H)$ is plotted, and the values are saved to BHloop.csv in the columns H_A_per_m, B_T.

BH.py — Core of the Magnetic Model

The BH.py file contains the entire physical and mathematical part of the Tellinen–Madelung algorithm. Functionally, it corresponds to BH.py in the [Chan–Engelhardt](#) project, but its internal structure is substantially different. It can conditionally be divided into several levels:



When BH.py is loaded, Major_Loop.csv is opened with the structure H_upper, B_upper, H_lower, B_lower, H_init, B_init. The first two pairs of columns are mandatory: (H_u, B_u) , (H_l, B_l) . They define the upper and lower branches of the major loop. The third pair (H_{init}, B_{init}) is desirable if it can be obtained experimentally. If such measurements are unavailable, the algorithm attempts to reconstruct the initial magnetization from the major branches. Each pair is read independently, so the number of experimental points on the three curves may differ. A header row is allowed in the file and is skipped automatically. After reading, the values of each branch are sorted by H , and the absence of duplicate field values is checked.

The discrete data are converted into continuous functions using [PchipInterpolator](#) from the SciPy library. The interpolated representations $C_u(H)$, $C_l(H)$, and, when corresponding data are available, $B_{init}(H)$, are obtained. PCHIP was chosen instead of an ordinary cubic spline because it preserves the shape of a monotonic dependence and is much less prone to producing artificial overshoot between experimental points.

In the code, these dependences correspond to the functions:

```
Upper (H)
Lower (H)
Binit (H)
```

If a separate initial curve is absent from Major_Loop.csv, $B_{init}(H) = \frac{C_u(H) + C_l(H)}{2}$ is used, calculated from the interpolated representations.

The function:

```
loop_width (H)
```

calculates $D(H) = C_u(H) - C_l(H)$. At the same time, the condition $D(H) > D_{tol}$, is checked because division by $D(H)$ enters directly into the Tellinen equations.

The function:

```
y_from_B (H, B)
```

calculates $y = \frac{B - C_l(H)}{C_u(H) - C_l(H)}$. It simultaneously performs a physical check: $0 \leq y \leq 1$. If the point lies significantly outside the major loop, the program generates an error. The inverse transformation is performed by the function:

```
B_from_y (H, y)
```

according to the formula $B = C_l(H) + yD(H)$.

The function:

```
_integral_f_dg(...)
```

calculates integrals of the general form $I = \int_a^b f(H) dg(H)$. Two specific integrals are used for Tellinen:

$I_u(a, b) = \int_a^b \frac{1}{D(H)} dC_u(H)$ and $I_l(a, b) = \int_a^b \frac{1}{D(H)} dC_l(H)$. They correspond to the functions:

```
integral_upper(a, b)
integral_lower(a, b).
```

The integration method is selected by the `IntegrationMethod` parameter in `Parameters.txt`. Two options are allowed:

```
trapezoid
lagrange
```

The first uses the previously derived formula $\approx \sum_i \frac{f_i + f_{i+1}}{2} (g_{i+1} - g_i)$ with second-order accuracy. The second uses three-point quadratic Lagrange interpolation with a global error of order $O(\Delta H^4)$. The number of intervals is specified by `IntegrationIntervals`.

The function:

```
tellinen_raw_y(H, start, direction)
```

implements the analytically integrated Tellinen equations. Let $start = (H_a, B_a)$. First $y_a = \frac{B_a - C_l(H_a)}{D(H_a)}$ is determined. For increasing field $direction = "inc"$, $y_T(H) = y_a \exp \left[- \int_{H_a}^H \frac{dC_u(h)}{D(h)} \right]$ is calculated. For decreasing field $direction = "dec"$, $y_T(H) = 1 - (1 - y_a) \exp \left[\int_{H_a}^H \frac{dC_l(h)}{D(h)} \right]$ is used.

The function:

```
tellinen_raw_branch(...)
```

converts the resulting value y_T back into magnetic flux density: $B_T(H) = C_l(H) + y_T(H)D(H)$. Thus, this function constructs the pure original Tellinen branch without Madelung memory.

If there are two or more real reversal points, the last one is the start of the current branch, $P_a = (H_a, B_a)$, while the preceding one — $P_b = (H_b, B_b)$ — is the Madelung return point.

The function:

```
tellinen_madelung_branch(
    H,
    start,
    target,
```

```
direction
```

```
)
```

first calculates the ordinary Tellinen branch $B_T(H)$ and its value at the target point $B_{Tb} = B_T(H_b)$. The scaling coefficient $A_B = \frac{B_b - B_a}{B_{Tb} - B_a}$ is then calculated. The final branch is defined as $B_{TM}(H) = B_a + A_B[B_T(H) - B_a]$. Thus, the branch shape is inherited from Tellinen, but it passes exactly through the Madelung return point: $B_{TM}(H_a) = B_a$, $B_{TM}(H_b) = B_b$. The condition $A_B > 0$ is additionally checked in the code. This is essential because $\frac{dB_{TM}}{dH} = A_B \frac{dB_T}{dH}$, and a positive coefficient preserves the direction of the original Tellinen branch. After the calculation, the following condition is also checked: $C_l(H) \leq B_{TM}(H) \leq C_u(H)$.

The function:

```
_advance(
    H,
    direction,
    root_branch,
    reversal_stack
)
```

is the central part of the Madelung-memory implementation. It analyzes the number of stored reversal points and selects which branch must be used. The input `reversal_stack` is never modified directly:

```
stack = list(reversal_stack)
```

A copy is created. This arrangement is particularly important for the future nonlinear solver, where the same accepted state must be reused repeatedly for trial values H . If the stack is empty, $reversal_stack = []$, there is no nested magnetic memory. Then $B = B_{root}(H)$. Depending on `root_branch`, this is: $B_{init}(H)$, $C_u(H)$, $C_l(H)$. If the stack contains one point $[P_1]$, this is the first return curve — FORC. There is not yet a preceding real reversal point, so the Madelung target is undefined. Therefore, the original Tellinen branch $B = B_T(H; P_1)$ is used directly. This rule applies regardless of whether P_1 was on the initial curve or on a major branch. With continued motion, such a Tellinen FORC gradually approaches the opposite major branch. For increasing H : $B_T(H) \rightarrow C_l(H)$, and for decreasing: $B_T(H) \rightarrow C_u(H)$. When the distance becomes smaller than B_{tol} , the first reversal point is considered exhausted, the stack is cleared, and the corresponding major branch becomes the new root: `lower_major` or `upper_major`. With two or more points present, the current branch is uniquely determined by the last two elements: $start = stack[-1]$, $target = stack[-2]$. Geometric consistency is checked first. For an increasing branch, $H_{target} > H_{start}$, is required, and for a decreasing branch $H_{target} < H_{start}$. Until the target is reached, the value B is calculated by the function:

```
tellinen_madelung_branch(...)
```

If the next prescribed value H reaches or passes the stored target point, the corresponding nested loop is considered closed. The last two points are removed from the stack:

```
stack.pop()
stack.pop()
```

that is, P_{start} and P_{target} . The function then does not terminate, but returns to the beginning of the while loop. This makes it possible to determine the restored parent branch automatically. If, after removal, $\text{len}(\text{stack}) \geq 2$, remains, an older Tellinen–Madelung branch is restored. If $\text{len}(\text{stack}) = 1$, the FORC is restored. If $\text{len}(\text{stack}) = 0$, the root branch is restored. Thus, one large numerical field step can close several nested loops sequentially. Schematically: $[P_1, P_2, P_3, P_4] \rightarrow [P_1, P_2] \rightarrow []$, with successive restoration of $P_4 \rightarrow P_3$, then $P_2 \rightarrow P_1$, and finally the root branch.

The functions `BH_increasing()` and `BH_decreasing()` form the external interface of `BH.py`, analogous to the functions of the same names in the [Chan–Engelhardt](#) project. For increasing field, the following is called:

```
BH_increasing(...)
```

and for decreasing field:

```
BH_decreasing(...)
```

If the direction of motion has not changed, the current stack is passed to `_advance()` unchanged. If a reversal has occurred, for example the motion was decreasing and then became increasing, the last accepted point $(H_{\text{last}}, B_{\text{last}})$ is added to the stack:

```
stack_new.append(
    (Hlast, Blast)
)
```

After this, `_advance()` is called for the new direction. Thus, `BH_increasing()` and `BH_decreasing()` perform only two operations:

1. detect and register a reversal point
2. pass the calculation of the current branch to the central `_advance()` function

This makes the structure of `BH.py` substantially simpler and more transparent than in Chan–Engelhardt.

Parameters.py

`Parameters.py` reads `Parameters.txt` and converts the values into numerical or string parameters. It must work not only with numbers because there are string parameters:

```
root_branch
IntegrationMethod
```

After reading, the required parameters and allowed values are checked. For example, `root_branch` can take only the values:

```
initial
upper_major
lower_major
```

and IntegrationMethod:

```
trapezoid
lagrange
```

The positivity of H_{tol} , B_{tol} , D_{tol} and the validity of IntegrationIntervals are also checked. If Major_Loop.csv already exists, Parameters.py additionally determines the admissible common range of the upper and lower major branches and checks: $H_{\text{min}} \leq H_{\text{start}} \leq H_{\text{max}}$. For root_branch=initial, when a separate experimental initial curve is present, membership of H_{start} in the range of this curve is also checked.

Parameters.txt

The file is intended to allow calculation parameters to be changed without editing the Python code. In the current version it contains four groups of data. The first is the parameter:

H_c

which is currently retained only because the unchanged Hscan2() function uses it when constructing the test signal. The second group defines the initial magnetic state:

```
root_branch=initial
Hstart=0.0
```

The third group specifies the numerical tolerances:

```
H_tol
B_tol
D_tol
```

The fourth group controls the calculation of the integrals:

```
IntegrationMethod
IntegrationIntervals
```

Thus, the material parameters themselves B_s , B_r , H_c are no longer parameters of the Tellinen model. The geometry of the actual material is supplied directly by Major_Loop.csv.

Hscan.py

The Hscan.py file is retained almost unchanged from the [Chan–Engelhardt](#) project. The function:

```
Hscan1(t)
```

creates a damped sinusoidal field dependence and is convenient for generating a large number of successively decreasing symmetric minor loops.

The function:

```
Hscan2(t)
```

creates a damped oscillation superimposed on a biased magnetic field and is intended for testing asymmetric nested loops. Retaining the same functions makes it possible to compare the Chan–Engelhardt and Tellinen–Madelung results directly for the same external-field history.

Major_Loop.csv

This file fundamentally distinguishes the new project from [Chan–Engelhardt](#). In Chan, the shape of the major loop was calculated analytically from the parameters B_s, B_r, H_c . In Tellinen–Madelung, the initial geometry is specified by tables:

```
H_upper, B_upper,
H_lower, B_lower,
H_init, B_init
```

This makes it possible to use experimentally measured curves directly. The number of points in each pair of columns may differ. The program reads each branch independently and then creates its continuous PCHIP-interpolated representation. For program debugging, Major_Loop.csv can be generated from the analytical Chan model. Subsequently, the same format allows the test loop to be replaced by actual experimental data without changing the Tellinen–Madelung algorithm.

Comparison of Calculations Using the Tellinen–Madelung and Chan–Engelhardt Models

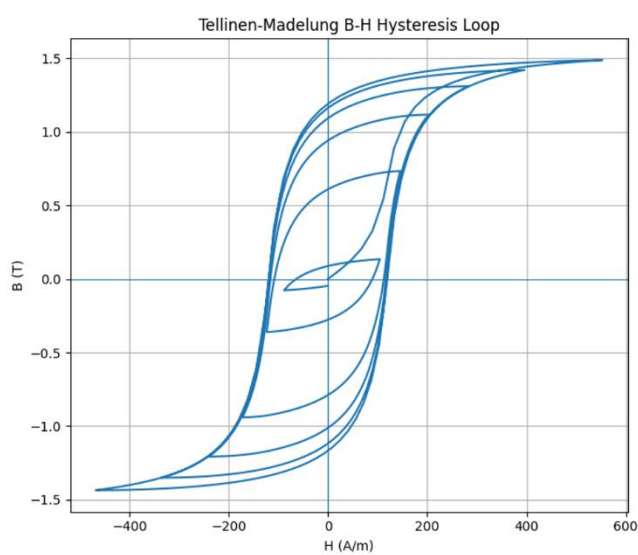
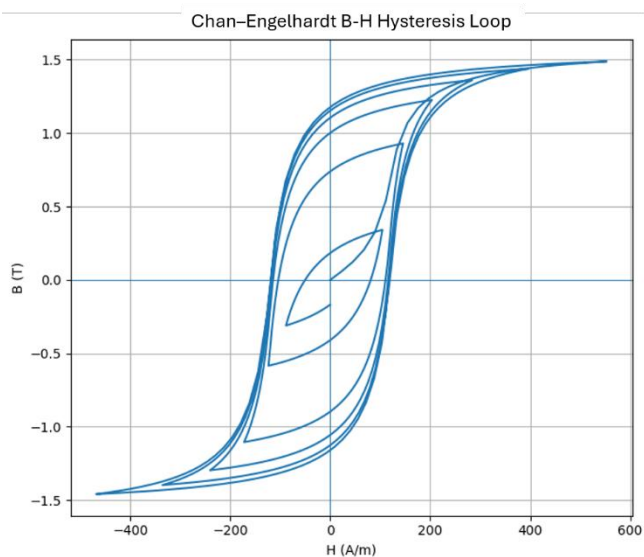
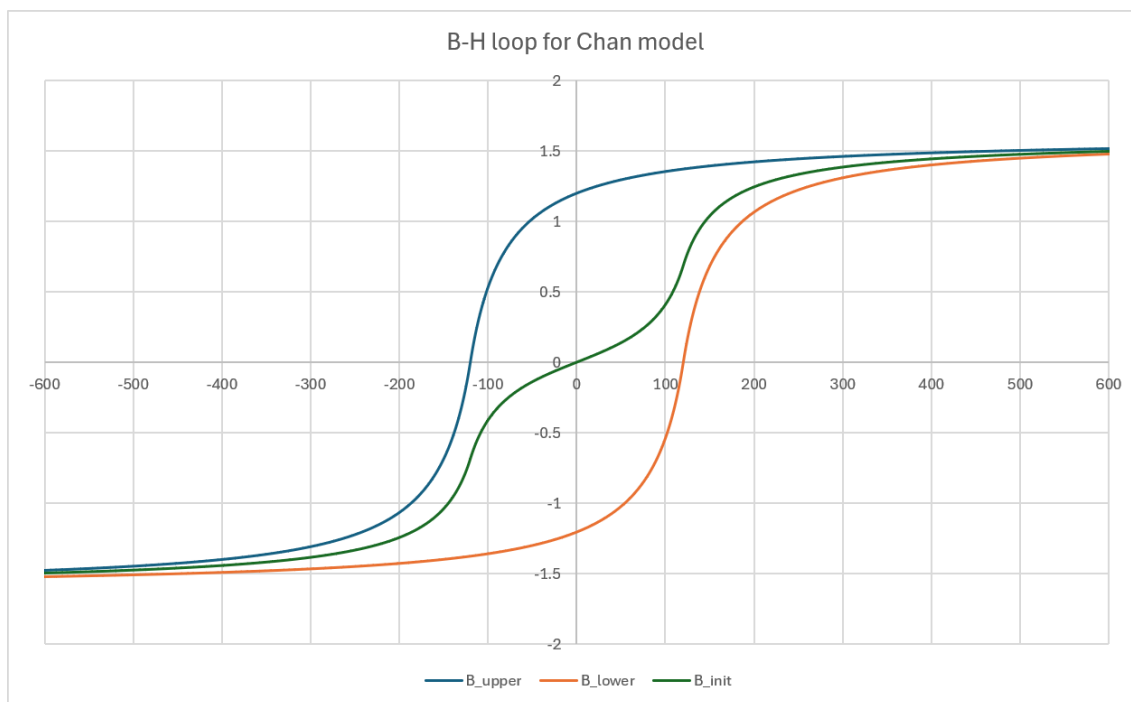
To compare the minor loops in the Tellinen–Madelung and Chan–Engelhardt models, we generated a test major loop using the Chan model [2] with the following parameters:

```
# Saturation magnetic flux density, T
Bs = 1.6

# Remanent magnetic flux density, T
Br = 1.2

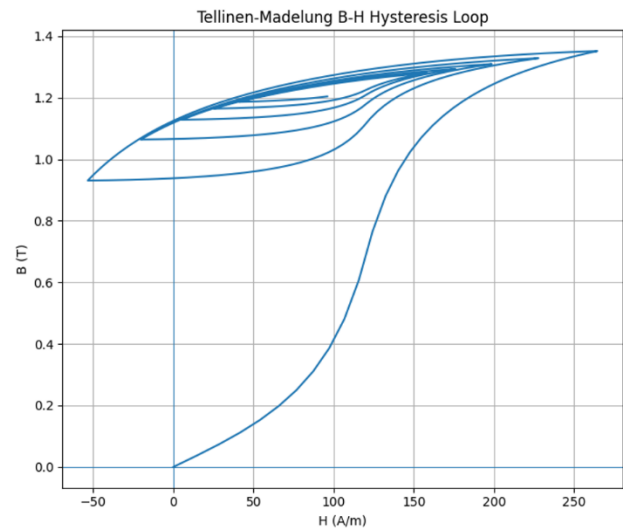
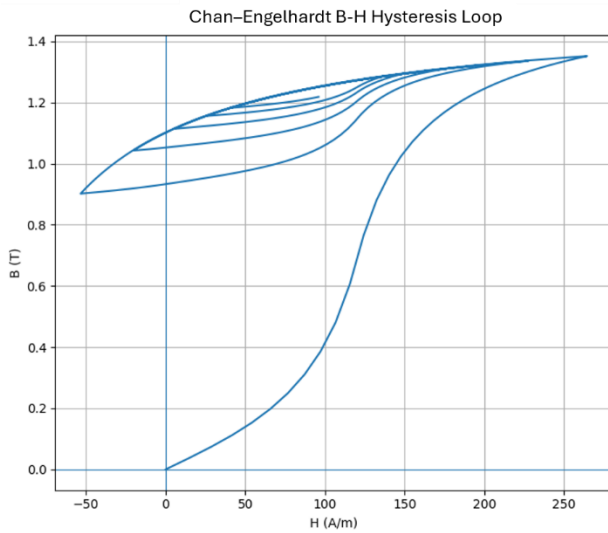
# Coercive field strength, A/m
Hc = 120.0

# Maximum absolute field in the generated table, A/m
Hmax = 600.0
```



$$H_{scan1}(t) = 600 \sin(120\pi t) \exp[-t/(3T)], \quad T = 2\pi/(120\pi) = 1/60 \text{ s}$$

A 60 Hz sinusoidal field with an initial amplitude of 600 A/m and an exponential decay time constant of $3T = 0.05$ s. Such a signal naturally produces a sequence of loops of decreasing amplitude around zero.



$$H_{scan2}(t) = H_{bias}(1 - e^{(-\tau/\tau_{bias})}) + A_0 e^{(-\tau/\tau_{decay})} \sin(\omega\tau)$$

In the current code, $H_{bias} = 0.8 \cdot H_c$, $A_0 = 1.5 \cdot H_c$, $n_{cycles} = 5$, $\tau_{bias} = 0.01 \cdot T_{sim}$, and $\tau_{decay} = 0.8 \cdot T_{sim}$, where T_{sim} is the simulation time. The purpose of H_{scan2} is to create several reversals around a positive bias.

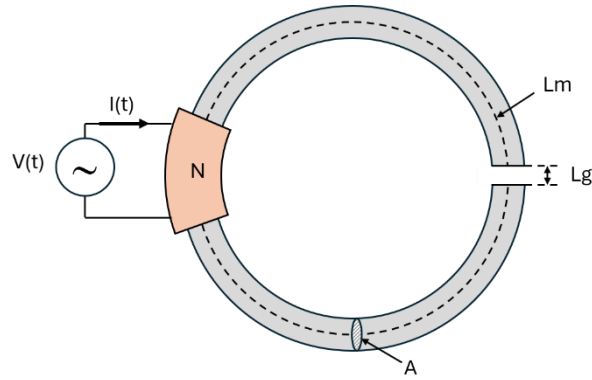
Overall, the behavior of the two models is similar, although it can be seen that the minor branches in the Tellinen–Madelung model are flatter. This follows directly from equations (4)–(7), in which the slope of the minor curves is determined by their distance from the major branches. This mechanism is absent from the Chan–Engelhardt model. The influence of magnetic memory is more difficult to trace. Specific excitations may be required to demonstrate this effect. Frankly, any radical differences between the two models would be rather concerning, since this would substantially complicate the choice of a hysteresis model for describing a real material.

Magnetic-Core Inductor Model

As the first practical application of the [Tellinen–Madelung](#) model [4], we consider the influence of hysteresis and minor-loop behavior on transient processes in an inductor with a ferromagnetic core. To assess the results obtained, we compare them with calculations in [LTspice](#), where the [Chan–Engelhardt](#) model is implemented.[3] Such a comparison will allow us to evaluate how suitable the Tellinen–Madelung model is for practical circuit-simulation tasks. LTspice calculations will be used as reference results because this simulator is widely used in engineering practice and its nonlinear magnetic models are well established for circuit-analysis problems. At the same time, LTspice results should not be regarded as an unconditional physical benchmark. The models used in it are known to have limitations that may become significant during rapid magnetization processes and for certain types of magnetic materials.[5] Therefore, the comparison should serve primarily as a means of mutual verification of the models and assessment of their applicability, rather than as unconditional validation of one model by the results of the other.

The fundamentals of magnetic-circuit theory are assumed to be known, so only those points that are directly required for constructing the computational algorithm are given here.

The inductor geometry is shown below. The shape of the core and its uniform cross-section may be arbitrary. We selected a ring. A gap in the core is also allowed.



Let us write the equations for the magnetic field strengths and the coil current $I(t)$, which are related to one another:

$$\begin{cases} H_m(t)L_m + H_g(t)L_g = NI(t) \\ RI(t) = V(t) - NA \frac{dB(t)}{dt} \end{cases} \quad (42)$$

where H_m is the magnetic field strength (A/m) inside the magnetic core, H_g is the magnetic field strength in the gap (A/m), R is the coil resistance $\Omega(\Omega)$, $V(t)$ is the external emf source (V), and $-NA \frac{dB(t)}{dt}$ is the self-induced emf of the coil. For the same core cross-section, the magnetic flux density $B(t)$ is uniformly distributed along it because of the continuity condition for its normal component at the interfaces. However, its dependence on the magnetic field strength must be taken into account. We choose $H_m(t)$ as the independent magnetic variable, so that $B(t) = B(H_m(t))$. Within the gap, we can immediately write $H_g(t) = B(H_m(t))/\mu_0$. As a result, we obtain the following system of equations:

$$\begin{cases} I(t) = \frac{L_m}{N} H_m(t) + \frac{L_g}{\mu_0 N} B(H_m(t)) \\ RI(t) = V(t) - NA \frac{dB(H_m(t))}{dt} \end{cases} \quad (43)$$

Substituting the current from the first equation into the second, we obtain a nonlinear differential equation for the magnetic field strength, closed by the hysteresis dependence $B(H_m(t))$:

$$NA \frac{dB(H_m(t))}{dt} + \frac{RL_m}{N} H_m(t) + \frac{RL_g}{\mu_0 N} B(H_m(t)) = V(t) \quad (44)$$

For the numerical solution of (44), we use the [Backward Euler](#) finite-difference method for the time step $t_{n+1} = t_n + \Delta t$:

$$NA \frac{B(H_{m,n+1}) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H_{m,n+1} + \frac{RL_g}{\mu_0 N} B(H_{m,n+1}) = V_{n+1} \quad (45)$$

where $H_{m,n} = H_m(t_n)$, $H_{m,n+1} = H_m(t_n + \Delta t)$, and $V_{n+1} = V(t_n + \Delta t)$.

For the numerical solution of (45), the residual function is introduced:

$$F(H) = NA \frac{B(H) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H + \frac{RL_g}{\mu_0 N} B(H) - V_{n+1}. \quad (46)$$

The required value of the magnetic field is determined from the condition $F(H_{m,n+1}) = 0$. After $H_{m,n+1}$ is found at the next step, the magnetic flux density and current are calculated:

$$B_{n+1} = B(H_{m,n+1}) \quad (47)$$

$$I_{n+1} = \frac{L_m}{N} H_{m,n+1} + \frac{L_g}{\mu_0 N} B(H_{m,n+1}) \quad (48)$$

The special feature of the problem is that the calculation of $B(H)$ is coupled to a hysteresis model with memory. The model state at time t_n can be denoted conventionally by S_n . It includes the current position on the hysteresis loop, the direction of the previous motion, and the reversal points. During the solution of the nonlinear equation using the residual $F(H)$, the state S_n must remain unchanged. Each trial field value H_{trial} is treated as an independent hypothesis for the system state at time t_{n+1} . For each such value, the complete hysteresis algorithm is executed: $(H_{\text{trial}}, S_n) \rightarrow (B_{\text{trial}}, S_{\text{trial}})$. The model may either continue along the current hysteresis branch or determine that a reversal occurred between steps n and $n + 1$ and move to the corresponding new branch. All transitions are performed in exactly the same way as in an ordinary sequential calculation of the hysteresis trajectory. However, the resulting state S_{trial} is only a trial state. It does not replace state S_n until the corresponding value H_{trial} satisfies the nonlinear equation. The next trial field value is again calculated from the same state S_n : $(H_{\text{trial},1}, S_n) \rightarrow (B_{\text{trial},1}, S_{\text{trial},1})$, $(H_{\text{trial},2}, S_n) \rightarrow (B_{\text{trial},2}, S_{\text{trial},2})$, and so on. A state obtained during one trial iteration must not be used as the initial state for the next trial iteration. Otherwise, the sequence of points generated by the root-finding method would itself become an artificial magnetic prehistory and would alter the hysteresis loop. Thus, during root finding the full hysteresis-model algorithm is executed, but all new parameters returned by it are retained only as trial values. Only after the final value $H_{m,n+1}$ has been found is the corresponding trial state accepted as the new physical state of the system: $S_{n+1} = S_{\text{trial}}$.

To apply the [bisection method](#) when finding the root of the nonlinear equation $F(H) = 0$, it is necessary to find an interval $[H_a, H_b]$, at whose endpoints the residual function has opposite signs: $F(H_a)F(H_b) \leq 0$. A natural first boundary is the known field value at the previous time step: $H_a = H_{m,n}$. In this case $B(H_a) = B(H_{m,n})$, so the derivative term in the residual function vanishes. Hence $F(H_{m,n}) = RI_n - V_{n+1}$. The quantity $V_{n+1} - RI_n$ determines the physically expected direction of change of the magnetic flux density. Indeed, from the electrical equation $NA \frac{dB}{dt} = V - RI$ it follows that for $V_{n+1} - RI_n > 0$ the magnetic flux density tends to increase, whereas for $V_{n+1} - RI_n < 0$ it tends to decrease. Therefore, the search for the second boundary is first performed in the corresponding direction from $H_{m,n}$. An initial search step ΔH_0 is specified. The first trial boundary is chosen as $H_b = H_{m,n} + s\Delta H_0$, where:

$$s = \begin{cases} +1, & V_{n+1} - RI_n > 0, \\ -1, & V_{n+1} - RI_n < 0. \end{cases}$$

At point H_b , the value $B(H_b)$ is calculated using the stored state S_n , after which $F(H_b)$ is determined. If condition $F(H_a)F(H_b) \leq 0$ is satisfied, the root is bracketed in the interval found and the bisection method can be used to locate it with the prescribed accuracy and maximum number of steps. If the residuals at the two endpoints have the same sign, the distance from $H_{m,n}$ to the second boundary is

doubled: $\Delta H_0, 2\Delta H_0, 4\Delta H_0, 8\Delta H_0, \dots$. Thus $H_b^{(j)} = H_{m,n} + s 2^j \Delta H_0$. At each such trial value, an independent hysteresis calculation is again performed from the same stored state S_n . If an interval with a sign change cannot be found in the physically expected direction, the search is repeated in the opposite direction to improve robustness. The same algorithm of successive doubling of the distance from $H_{m,n}$ is used. This additional search is particularly useful near hysteresis-loop reversal points, where a simple estimate of the direction from $V_{n+1} - RI_n$ may be insufficient.

After an interval $[H_a, H_b]$, satisfying condition $F(H_a)F(H_b) \leq 0$, has been found, the [bisection method](#) itself begins. At each iteration, the midpoint of the interval is calculated: $H_c = \frac{H_a + H_b}{2}$. For this trial value, starting from the unchanged state S_n , the full hysteresis-model calculation is performed and $B_c = B(H_c)$, is determined, after which the residual $F_c = F(H_c)$ is calculated. If $F(H_a)F(H_c) \leq 0$, the root lies in interval $[H_a, H_c]$, and $H_b = H_c$ is adopted. If $F(H_a)F(H_c) > 0$, the root lies in interval $[H_c, H_b]$, and $H_a = H_c$ is adopted. The procedure is then repeated. The search terminates when one of the convergence conditions is satisfied: $|F(H_c)| \leq \varepsilon_F$, or $|H_b - H_a| \leq \varepsilon_H$, where ε_F is the allowable residual of the nonlinear equation and ε_H is the allowable width of the search interval in magnetic field strength. If the required accuracy is not achieved within the specified maximum number of iterations, the time step is considered unsolved. In this case Δt should be reduced, the bracketing parameters changed, or the magnetic and electrical model parameters checked. The last approximation should not be accepted automatically because it may not satisfy the original equation to the required accuracy.

After the root is found, $H_{m,n+1} = H_c$ is accepted. At the same time, the values (47) and (48) corresponding to this root are accepted, and the trial state of the hysteresis model becomes the new accepted state: $S_{n+1} = S_{\text{trial}}$. Only after this is the time step considered complete, and state $H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1}$ is used as the initial state when advancing to the next time instant. The complete algorithm can therefore be represented by the following sequence:

$$\boxed{(H_{m,n}, B_n, I_n, S_n) \rightarrow V_{n+1} \rightarrow \text{interval search} \rightarrow F(H_{m,n+1}) = 0 \rightarrow (H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1})}$$

The fundamental feature of the method is that all trial calculations within the root search are performed with the initial hysteresis state S_n held unchanged, and the new state is committed only after the nonlinear equation has been solved finally.

Algorithmic Implementation of the Inductor Calculation

The special feature of the problem is that the calculation of $B(H)$ is coupled to a hysteresis model with memory. The model state at time t_n can be denoted conventionally by S_n . It includes the current position on the hysteresis loop, the direction of the previous motion, and the reversal points. During solution of the nonlinear equation using the residual $F(H)$, state S_n must remain unchanged. Each trial field value H_{trial} is treated as an independent hypothesis for the system state at time t_{n+1} . For each such value, the complete hysteresis algorithm is executed: $(H_{\text{trial}}, S_n) \rightarrow (B_{\text{trial}}, S_{\text{trial}})$. The model may either continue along the current hysteresis branch or determine that a reversal occurred between steps n and $n + 1$ and move to the corresponding new branch. All transitions are performed in exactly the same way as in an ordinary sequential calculation of the hysteresis trajectory. However, the

resulting state S_{trial} is only a trial state. It does not replace state S_n until the corresponding value H_{trial} satisfies the nonlinear equation. The next trial field value is again calculated from the same state S_n : $(H_{\text{trial},1}, S_n) \rightarrow (B_{\text{trial},1}, S_{\text{trial},1})$, $(H_{\text{trial},2}, S_n) \rightarrow (B_{\text{trial},2}, S_{\text{trial},2})$, and so on. A state obtained during one trial iteration must not be used as the initial state for the next trial iteration. Otherwise, the sequence of points generated by the root-finding method would itself become an artificial magnetic prehistory and would alter the hysteresis loop. Thus, during root finding the full hysteresis-model algorithm is executed, but all new parameters returned by it are retained only as trial values. Only after the final value $H_{m,n+1}$ has been found is the corresponding trial state accepted as the new physical state of the system: $S_{n+1} = S_{\text{trial}}$.

To apply the bisection method when finding the root of nonlinear equation (46), it is first necessary to find an interval $[H_a, H_b]$ at whose endpoints the residual function has opposite signs: $F(H_a)F(H_b) \leq 0$. A natural starting point is the known field value at the previous time step H_n . For $H_{\text{trial}} = H_n$, the magnetic flux density coincides with B_n , so the derivative term in (46) vanishes. The quantity $\text{drive} = V_{n+1} - RI_n$ specifies the physically expected direction of the first search: for $\text{drive} > 0$, values $H > H_n$ are examined first, whereas for $\text{drive} < 0$ — $H < H_n$. This direction is used only as the first assumption and does not restrict the subsequent search.

The search for the second interval boundary is carried out in two stages. First, starting from H_n , successive advancement is performed with the initial step `initial_H_step`; after each unsuccessful check, the step is multiplied by `bracket_growth_factor`. A change in the sign of the residual is monitored between each pair of adjacent trial points. The search is limited to the common range $H_{\min} \leq H \leq H_{\max}$ defined by `Major_Loop.csv`. If the expanding search reaches the corresponding boundary without detecting a sign change, a second, denser pass is performed: the entire available interval from H_n to the boundary is divided into `bracket_scan_points` parts and checked sequentially. If no bracket is found after this either, the same two-stage search is repeated completely in the opposite direction. If no sign-changing interval exists, calculation of the time step is terminated with a diagnostic message; magnetic data are not extrapolated beyond the range of `Major_Loop.csv`.

After an interval $[H_a, H_b]$, satisfying condition $F(H_a)F(H_b) \leq 0$, has been found, the [bisection method](#) itself begins. At each iteration the midpoint of the interval is calculated: $H_c = \frac{H_a + H_b}{2}$. For this trial value, starting from the unchanged state S_n , the full hysteresis-model calculation is performed and $B_c = B(H_c)$, is determined, after which the residual $F_c = F(H_c)$ is calculated. If $F(H_a)F(H_c) \leq 0$, the root lies in interval $[H_a, H_c]$, and $H_b = H_c$ is adopted. If $F(H_a)F(H_c) > 0$, the root lies in interval $[H_c, H_b]$, and $H_a = H_c$ is adopted. The procedure is then repeated. The search terminates when one of the convergence conditions is satisfied: $|F(H_c)| \leq \varepsilon_F$, or $|H_b - H_a| \leq \varepsilon_H$, where ε_F is the allowable residual of the nonlinear equation and ε_H is the allowable width of the search interval in magnetic field strength. If the required accuracy is not achieved within the specified maximum number of iterations, the time step is considered unsolved. In this case Δt should be reduced, the bracketing parameters changed, or the magnetic and electrical model parameters checked. The last approximation should not be accepted automatically because it may not satisfy the original equation to the required accuracy.

After the root is found, $H_{m,n+1} = H_c$ is accepted. At the same time, the values (47) and (48) corresponding to this root are accepted, and the trial state of the hysteresis model becomes the new

accepted state: $S_{n+1} = S_{\text{trial}}$. Only after this is the time step considered complete, and state $H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1}$ is used as the initial state when advancing to the next time instant. The complete algorithm can therefore be represented by the following sequence:

$$\boxed{(H_{m,n}, B_n, I_n, S_n) \rightarrow V_{n+1} \rightarrow \text{interval search} \rightarrow F(H_{m,n+1}) = 0 \rightarrow (H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1})}$$

1. Accepted state at the beginning of the step. At time t_n , H_n, B_n, I_n and the magnetic state S_n , including previous_direction, root_branch, and reversal_stack, are known. For the next time instant $V_{n+1} = V(t_{n+1})$ is calculated.

2. Trial field value. Any H_{trial} is considered only as a hypothesis for the system state at time t_{n+1} . For it, the full Tellinen–Madelung calculation is performed from a copy of the accepted state S_n , yielding B_{trial} and the trial state S_{trial} .

3. Trial current and residual. The current I_{trial} is calculated from the magnetic-circuit equation, after which $F(H_{\text{trial}})$ is determined from (46). Thus, each H_{trial} corresponds to a consistent set $H_{\text{trial}}, B_{\text{trial}}, I_{\text{trial}}, F(H_{\text{trial}}), S_{\text{trial}}$.

4. Selection of the first search direction. The physically expected direction of field change is selected from the sign of drive $= V_{n+1} - RI_n$. At this point only the order of the search is determined; the final direction is still established by the solution of the nonlinear equation itself.

5. Bracketing search. In the selected direction, an expanding search is performed with initial_H_step and the multiplier bracket_growth_factor. A sign change of F is monitored between adjacent trial points. All trial values are calculated from the same state S_n .

6. Additional scan and opposite direction. If the expanding search does not produce a bracket, the admissible interval H is scanned more densely using bracket_scan_points subdivisions. If this fails, the same procedure is repeated in the opposite direction. If $F(H_a)F(H_b) \leq 0$ cannot be obtained, the time step is considered unsolved.

7. Bisection iteration. For the interval found, $H_m = \frac{H_a + H_b}{2}$ is calculated. The values B_m, I_m and the trial magnetic state S_m are again calculated directly from S_n , after which $F(H_m)$ is determined.

8. Interval reduction. From the signs $F(H_a), F(H_m), F(H_b)$, the half of the interval in which the residual sign change remains is retained. A new midpoint is then calculated and the procedure is repeated.

9. Convergence criterion. The iterative search terminates when at least one of the conditions $|F(H_m)| \leq \text{residual_tol}$ or $|H_b - H_a| \leq H_{\text{tol}}$ is satisfied. If the required accuracy is not achieved within max_bisection_iter, the last approximation is not accepted automatically.

10. Acceptance of the solution. After the root has been found, H_{n+1} , the corresponding B_{n+1} and I_{n+1} , and exactly the trial state S_{n+1} obtained for the accepted field value are accepted.

11. Committing magnetic memory. Only after the root has been accepted are root_branch, reversal_stack, and previous_direction replaced by their new values. All other trial states arising during bracketing and bisection are discarded.

12. Advance to the next time step. The accepted H_{n+1} , B_{n+1} , I_{n+1} and S_{n+1} become the sole initial state for calculating time t_{n+2} . Thus, the internal sequence of trial points generated by the iterative solver does not become an artificial magnetic prehistory.

The fundamental feature of the method is that all trial calculations within the root search are performed with the initial hysteresis state S_n held unchanged, and the new state is committed only after the nonlinear equation has been solved finally.

Structure of the Python Project for Inductor Calculation

After the hysteresis model has been constructed, the next project uses it as the magnetic core while solving the nonlinear inductor problem. The theoretical formulation of this problem is given above in equations (42)–(48). In the software implementation, the magnetic field strength H is no longer prescribed in advance by $H_{\text{scan}}(t)$, but is determined at each time step from the electrical equation together with the hysteresis dependence $B(H)$.

The project structure is as follows:

```
Tellinen-Madelung-Model-Inductor/
├── main.py
├── BH.py
├── Parameters.py
├── Parameters.txt
├── V.py
├── Major_Loop.csv
├── Current.csv
├── BHloop.csv
└── State.csv
```

The BH.py file contains the Tellinen–Madelung model described above and in this project serves as an independent magnetic module. An important feature of its implementation is that the passed reversal_stack is not modified directly: a copy is created before calculation. Therefore, the same accepted magnetic state can be used repeatedly for the trial values H that arise while solving the nonlinear equation. The external interface of the magnetic model remains the functions BH_increasing() and BH_decreasing(), which receive the accepted H_{last} , B_{last} , root_branch, and reversal_stack and return the calculated flux density together with a new trial memory state. Thus, the principal difference between the new project and the direct $B(H)$ -construction program lies in main.py. Previously, the sequence of H values were prescribed externally and the magnetic model only calculated the corresponding B values. In the inductor problem, the unknown at each time step is H_{n+1} , which is determined from the nonlinear equation by the bisection method. The BH.py module itself is therefore used almost as an independent function with memory: $(H_{\text{trial}}, S_n) \rightarrow (B_{\text{trial}}, S_{\text{trial}})$, where S_n denotes the accepted magnetic state at the beginning of the next time step.

main.py — Main Inductor Calculation Module

At the beginning of main.py, the time grid is formed. The number of steps is determined by the ratio $\text{simulation_time}/dt$, and the program additionally checks that the total simulation time is an integer number of time steps. The initial magnetic state is then specified. The value H_0 is taken from the Hstart parameter, while root_branch determines on which of the three possible root trajectories the material is located: initial, upper_major, or lower_major. The initial flux density is not specified independently, but is calculated by the function:

```
root_branch_value(Hlast, root_branch)
```

The initial current is then determined directly from the magnetic-circuit equation: $I_0 = \frac{L_m H_0 + (L_g/\mu_0) B_0}{N}$. Therefore, the initial H_0 , B_0 , and I_0 are always mutually consistent. At the same time an empty reversal_stack is created, and previous_direction is set in accordance with the selected root branch and initial field.

The main feature of the program is that the hysteresis model is called repeatedly within a single time step. For this purpose, the function is used:

```
evaluate_hysteresis_trial(Htrial)
```

It treats H_{trial} as a trial field value at time t_{n+1} . If it practically coincides with the accepted H_{last} , B_{last} and a copy of the current memory state are returned. For $H_{\text{trial}} > H_{\text{last}}$, BH_increasing() is called, and for $H_{\text{trial}} < H_{\text{last}}$, BH_decreasing() is called. Both functions always receive the same accepted state from the preceding time step. The resulting root_trial and stack_trial are regarded exclusively as trial values and do not yet modify the physical magnetic history.

For each trial field value, residual() first obtains B_{trial} and the trial magnetic state, and then calculates the current: $I_{\text{trial}} = \frac{L_m H_{\text{trial}} + (L_g/\mu_0) B_{\text{trial}}}{N}$. The residual of the discretized electrical equation is then calculated: $F(H_{\text{trial}}) = R I_{\text{trial}} + N A \frac{B_{\text{trial}} - B_{\text{last}}}{\Delta t} - V_{\text{next}}$. Thus, each trial value H_{trial} corresponds to a complete consistent set $H_{\text{trial}}, B_{\text{trial}}, I_{\text{trial}}, S_{\text{trial}}, F(H_{\text{trial}})$. The residual() function is precisely what couples the electrical part of the problem to the magnetic-hysteresis model.

Bracketing for the Bisection Method

Before applying the bisection method, values H_a and H_b must be found such that: $F(H_a)F(H_b) \leq 0$. This operation is implemented in a separate function:

```
_find_bracket_in_direction()
```

The accepted value H_{last} is always used as the starting point. The search is initially performed in only one direction. If this is the increasing direction, the upper admissible boundary is H_{max} from Major_Loop.csv. For the decreasing direction, H_{min} is used. Thus, the program does not extrapolate the experimental magnetic dependences beyond the region defined by the input data. Bracketing is performed in two stages. In the first stage, successive advancement starts from H_{last} with the initial step initial_H_step. If no residual sign change is detected, the next increment is multiplied by bracket_growth_factor. In the current version: $\text{bracket_growth_factor} = 1.6$. The residual sign is

compared between every pair of adjacent trial points: $F(H_{k-1})F(H_k) \leq 0$. This is substantially more careful than comparing all subsequent points only with H_{last} , because it allows a local sign change to be detected even for a more complex dependence $F(H)$.

If the expanding search reaches the boundary of the admissible range H without detecting a sign change, the second stage is performed. The entire interval between H_{last} and the corresponding boundary of Major_Loop.csv is divided into `bracket_scan_points` equal parts and checked sequentially with a smaller step. In the current version: `bracket_scan_points` = 200. This additional scan reduces the probability of missing a narrow residual sign-change interval, for example near a reversal point or on a section with rapidly varying differential permeability.

The physically preferred initial search direction is determined by the quantity: $drive = V_{\text{next}} - RI_{\text{last}}$. If $drive > 0$, $H > H_{\text{last}}$ is examined first; if $drive < 0$, the search begins in the direction of decreasing H . However, this condition is used only to choose the first search direction. If a bracket cannot be found there, the entire search is repeated in the opposite direction. If no sign-changing interval is found in either direction, the program does not continue with an arbitrary approximation. Instead, the residual values at both boundaries of the admissible range are calculated and a diagnostic message is issued containing H_{last} , B_{last} , I_{last} , V_{next} , $drive$, $F(H_{\text{min}})$, $F(H_{\text{last}})$, and $F(H_{\text{max}})$. This makes it possible to distinguish a numerical search problem from the physical situation in which the required solution actually lies outside the range specified by Major_Loop.csv.

Solution of the Nonlinear Equation

After the interval $[H_a, H_b]$ has been found, the standard bisection method begins. At each iteration, $H_m = \frac{H_a + H_b}{2}$ is calculated. For H_m , `residual()` is called again, meaning that the full trial calculation of the Tellinen–Madelung model is repeated, B_m , I_m , the corresponding trial magnetic-memory state, and $F(H_m)$ are calculated. If $F(H_a)F(H_m) \leq 0$, the new right boundary $H_b = H_m$ is accepted. Otherwise $H_a = H_m$ is set. The key point remains that the magnetic state from one bisection iteration is not passed to the next iteration. Each H_m is calculated from the same accepted state at time t_n . The calculation terminates if at least one of the conditions $|F(H_m)| \leq \text{residual_tol}$ or $|H_b - H_a| \leq H_{\text{tol}}$ is satisfied. At the same time, the point with the smallest absolute residual is stored during the search. If the required accuracy is not achieved within `max_bisection_iter`, the time step is considered unsolved and the program terminates with an error message rather than automatically accepting the last approximation.

Committing the Accepted State

After `solve_time_step()` has completed successfully, H_{next} , B_{next} , I_{next} , are returned, together with `root_next`, `stack_next`, and the direction of motion corresponding exactly to this root. Only at this point:

```
root_branch = root_next
```

and

```
reversal_stack = stack_next
```

Thus, all intermediate states arising during bracket search and bisection are discarded. If the new field actually differs from the previous one by more than H_{tol} , `previous_direction` is also updated. After this, the new H , B , and I become accepted values and are used at the next time step. This separation between accepted and trial states is fundamentally important for the joint operation of the nonlinear solver and the Madelung model. The sequence of trial H values generated by the numerical method is not a real time trajectory of the magnetic field and therefore must not create false reversal points or erase the existing magnetic memory.

Parameters.py and Parameters.txt

The `Parameters.py` file performs the same auxiliary function as in the direct hysteresis-loop calculation project: it reads `Parameters.txt`, distinguishes numerical and string parameters, and checks their validity. `root_branch` and `IntegrationMethod` remain string parameters. All other parameters are converted to numerical form. For the inductor project, the presence of electrical-circuit, time-solver, and root-search parameters is checked additionally.

In addition to the Tellinen–Madelung parameters already described, `Parameters.txt` contains the magnetic-circuit parameters:

```
N, R, Lm, A, Lg
```

external-voltage parameters:

```
V_offset, V_amplitude, V_frequency
```

time-integration parameters:

```
simulation_time, dt
```

as well as the nonlinear-solver parameters:

```
residual_tol, max_bracket_iter, max_bisection_iter, initial_H_step,  
bracket_growth_factor, bracket_scan_points
```

The parameter `bracket_growth_factor` must be greater than one, and `bracket_scan_points` must be at least two; these conditions are checked separately when the file is read.

V.py

Instead of the `Hscan.py` file used previously, the inductor project introduces `V.py`. It specifies not the magnetic-field history, but the external voltage applied to the winding. In the current implementation, the function is used: $V(t) = V_{\text{offset}} + V_{\text{amplitude}}\sin(2\pi V_{\text{frequency}}t)$. All source parameters are stored in `Parameters.txt`, so the excitation waveform can be changed independently of the main calculation algorithm.

Output Files

At the end of the calculation, main.py creates three output files. Current.csv contains two columns:

```
time_s, I_A
```

and represents the calculated transient current in the winding. BHloop.csv contains:

```
H_A_per_m, B_T
```

and stores the actual magnetic trajectory of the core, which arises not from a prescribed $H(t)$ but as the result of solving the complete electromagnetic problem.

An additional diagnostic file State.csv is also generated:

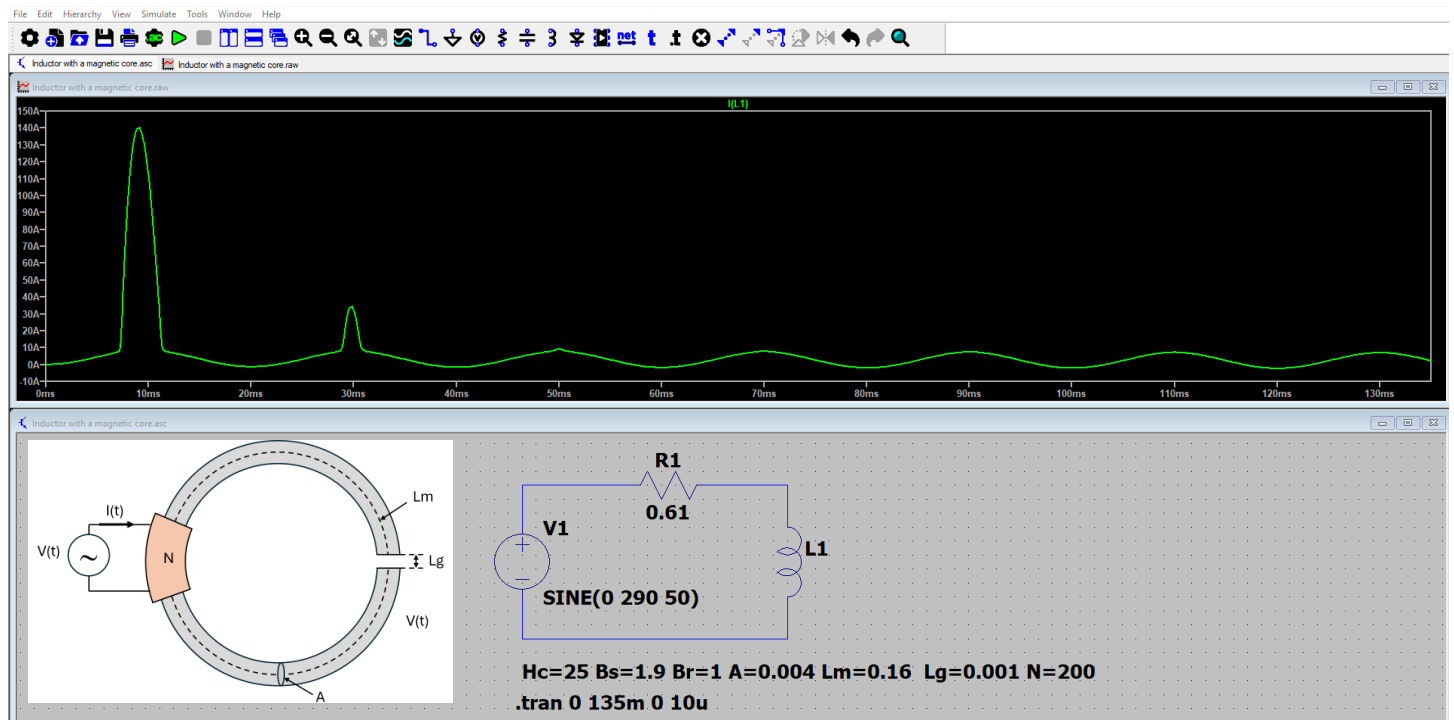
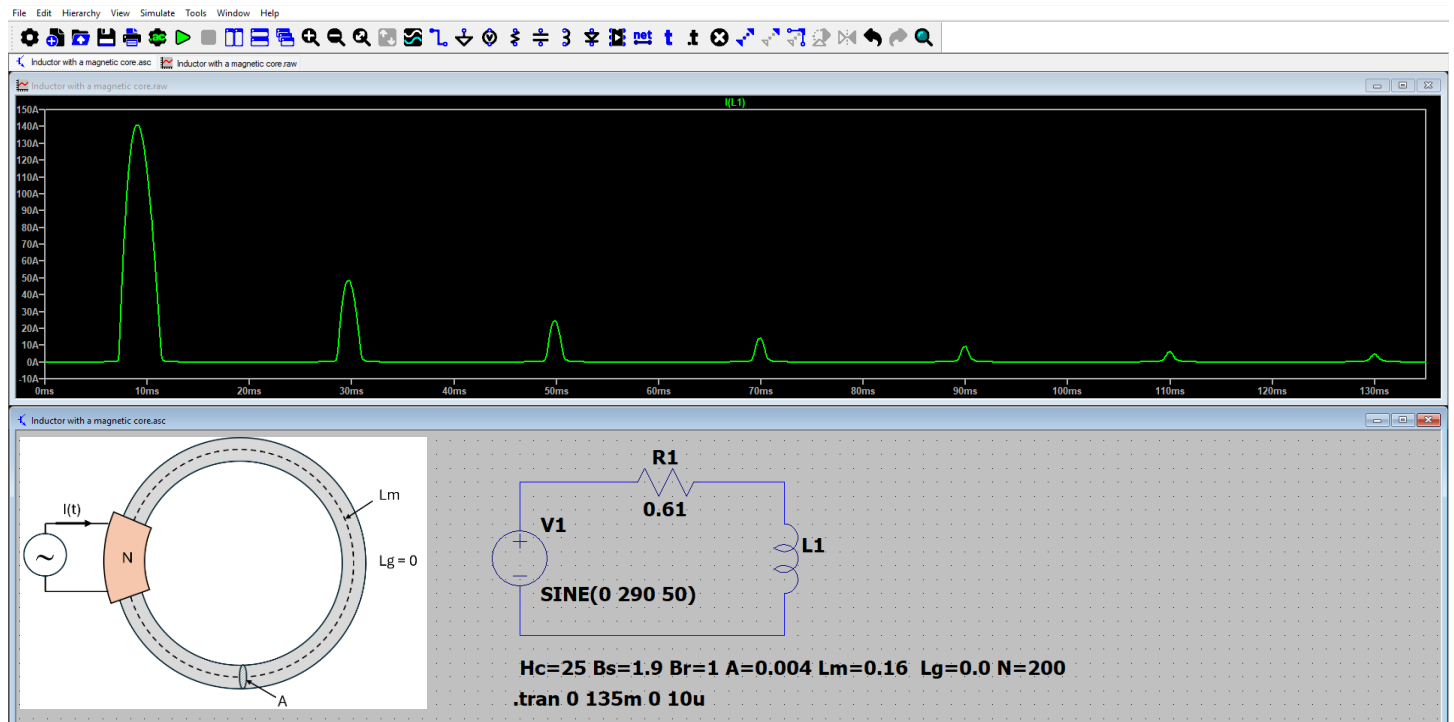
```
time_s, root_branch, stack_depth
```

It makes it possible to trace changes in the root-branch type and the depth of the Madelung stack over time. The list of reversal points itself is not written to the file: stack_depth shows only the current number of active magnetic-memory points.

At the same time, the program plots two graphs: the transient current $I(t)$ and the complete trajectory $B(H)$. As a result, the program structure retains the same separation of functions used in the hysteresis-loop project. Major_Loop.csv specifies the properties of the magnetic material, BH.py is responsible only for the Tellinen–Madelung law and magnetic memory, V.py specifies the external excitation, Parameters.txt contains the magnetic-circuit and numerical-calculation parameters, and main.py solves the nonlinear inductor equation and determines the physically accepted sequence of magnetic states. This separation makes it possible to change the material model, electrical parameters, or excitation law independently of one another without restructuring the main algorithm.

Comparison with LTspice Inductor Simulations

We tested the Tellinen–Madelung model for the inductor circuit used in patent [US 7502723B1](#) [6] to demonstrate the advantage of the Chan–Engelhardt model [3] over the original Chan model [2]. The Chan–Engelhardt model is implemented in LTspice. The LTspice inductor simulations are shown below; they calculate the transient current through an inductor with a small coil resistance 0.61Ω and $N = 200$ turns when a sinusoidal generator is switched on. The subsequent steady sinusoidal oscillations lie outside the time interval considered and are not shown. The first graph is for a magnetic core without a gap, and the second is for a gapped core. The geometric and magnetic core parameters are shown in the circuit diagrams: A is the core cross-sectional area (m^2), L_m is the mean core length (m), L_g is the gap length (m), H_c is the coercive field strength (A/m), B_r is the remanent flux density (T), and B_s is the saturation flux density (T). The graph shows the current (A) through the inductor as a function of time (s). The sinusoidal voltage source had amplitude 290 V, frequency 50 Hz, and zero phase, i.e. it started from zero at time $t = 0$. Thus, the magnetization simulation began from the fully demagnetized state. The $I(t)$ graph shows the current (A) through the inductor versus time (s). The LTspice files are stored in the project folder “LTspice_Inductor with a magnetic core” in the [GitHub](#) repository.[4]



Our calculations agreed completely with the LTspice simulations, as shown in the figures below. The Python algorithm used the following parameters:

```
# =====
# INITIAL MAGNETIC STATE
# =====

# root_branch:
#   initial
#   upper_major
#   lower_major
root_branch=initial

# Initial magnetic field strength [A/m]
Hstart=0.0

# =====
# MAGNETIC CORE AND COIL PARAMETERS
# =====

# Number of coil turns [-]
N=200

# Coil resistance [ohm]
R=0.61

# Mean magnetic path length in ferromagnetic core [m]
Lm=0.16

# Effective magnetic cross-sectional area [m^2]
A=0.004

# Air-gap length [m]
Lg=0.0

# =====
# VOLTAGE SOURCE PARAMETERS
# =====

# DC voltage offset [V]
V_offset=0.0

# Sinusoidal voltage amplitude [V]
V_amplitude=290.0

# Voltage frequency [Hz]
V_frequency=50.0

# =====
# SIMULATION PARAMETERS
# =====

# Total simulation time [s]
simulation_time=0.135

# Time step [s]
# 1e-5 gives 2000 steps per 50-Hz period.
# Reduce it for a time-step convergence study.
dt=0.00001

# =====
# TELLINEN-MADELUNG NUMERICAL PARAMETERS
# =====

# H comparison tolerance [A/m]
H_tol=1.0e-10

# B comparison / envelope tolerance [T]
B_tol=1.0e-9

# Minimum admissible major-loop width [T]
D_tol=1.0e-12
```

```

# Numerical Stieltjes integral method:
#   trapezoid
#   lagrange
IntegrationMethod=lagrange

# Number of integration intervals per Tellinen integral
IntegrationIntervals=200

# =====
# NONLINEAR SOLVER PARAMETERS
# =====

# Electrical residual tolerance [V]
residual_tol=1.0e-9

# Maximum number of root-bracketing iterations
max_bracket_iter=80

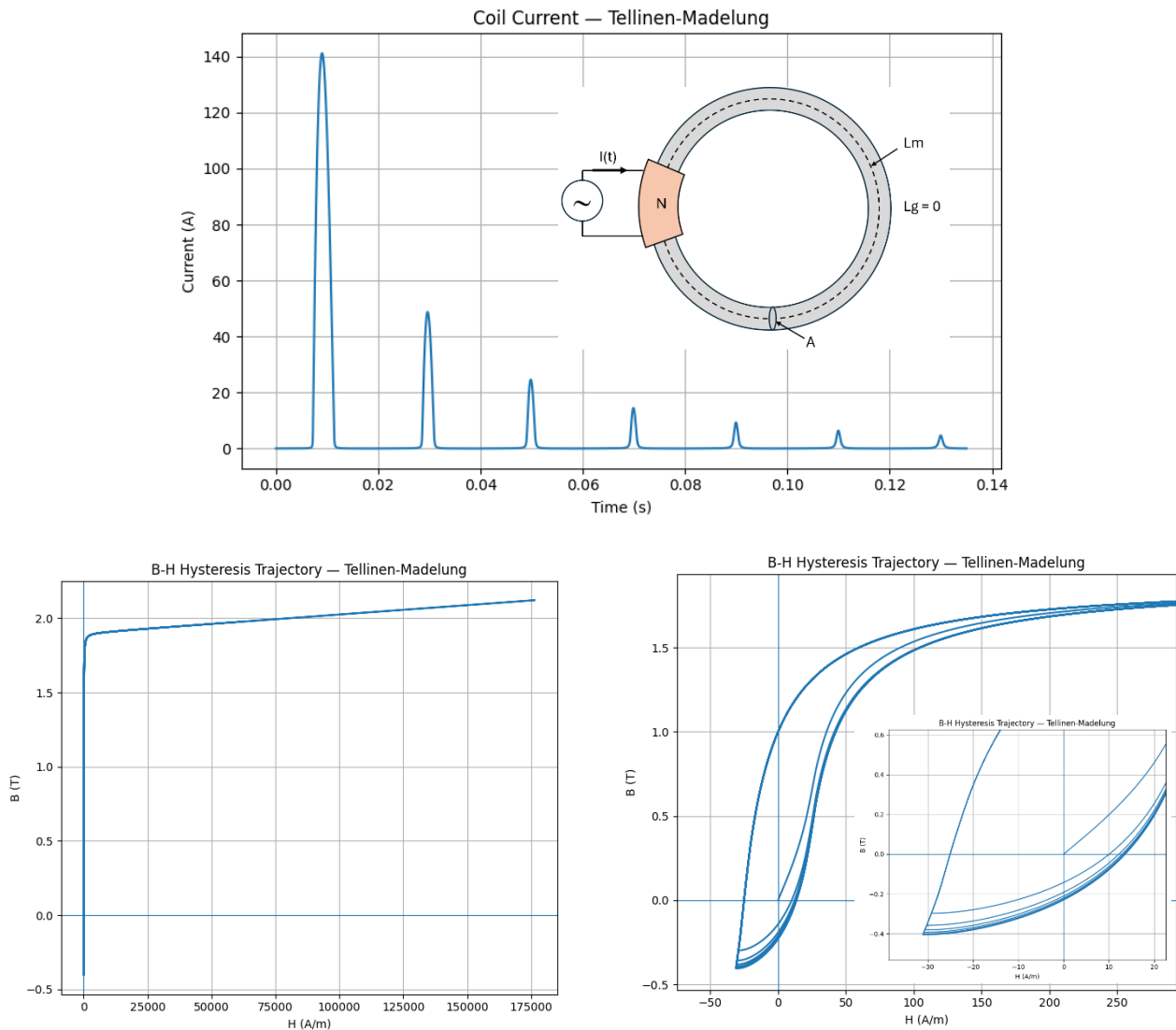
# Maximum number of bisection iterations
max_bisection_iter=120

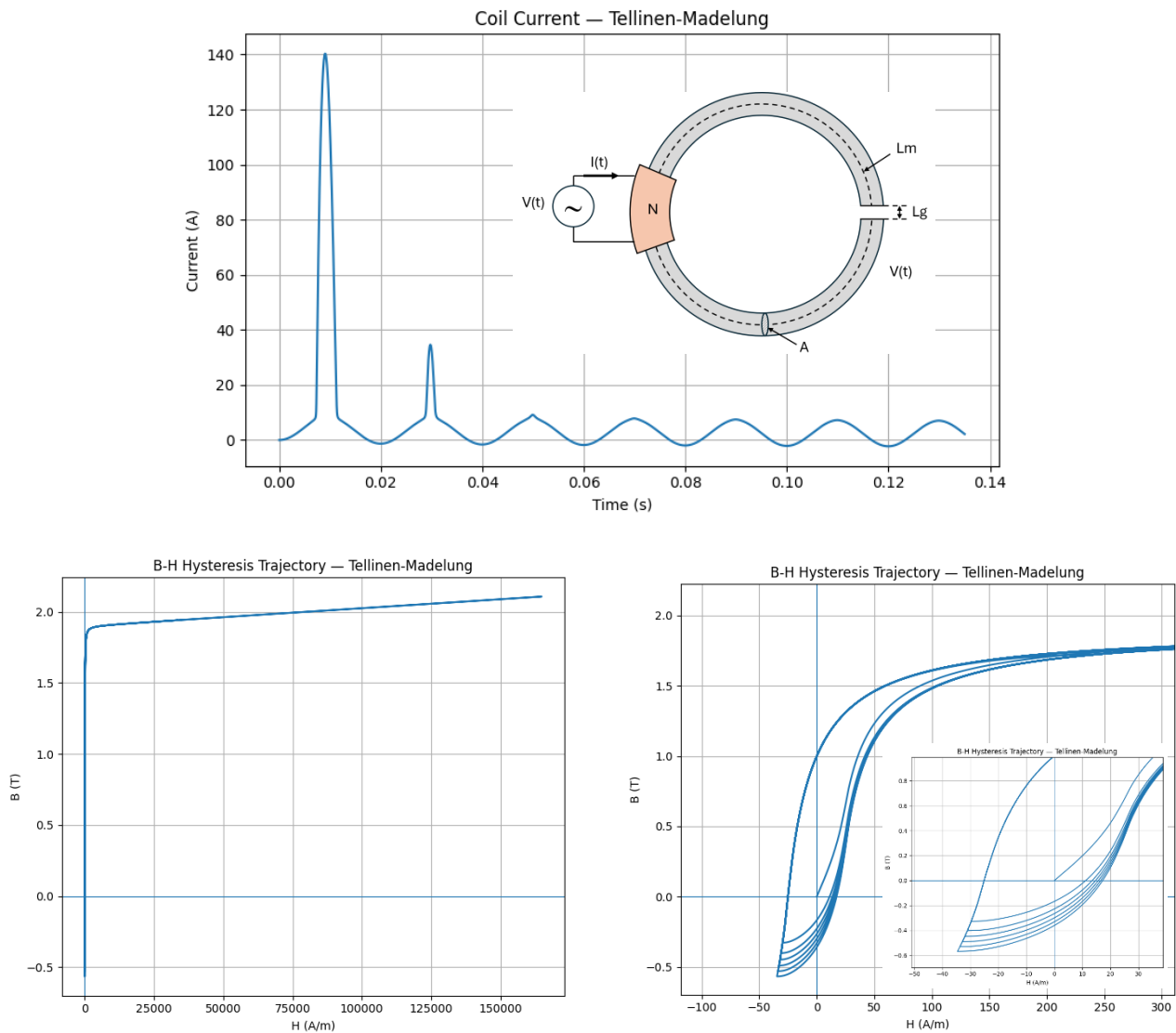
# Initial H increment for root bracketing [A/m]
initial_H_step=6.0

# Multiplicative growth factor for the outward bracket search
bracket_growth_factor=1.6

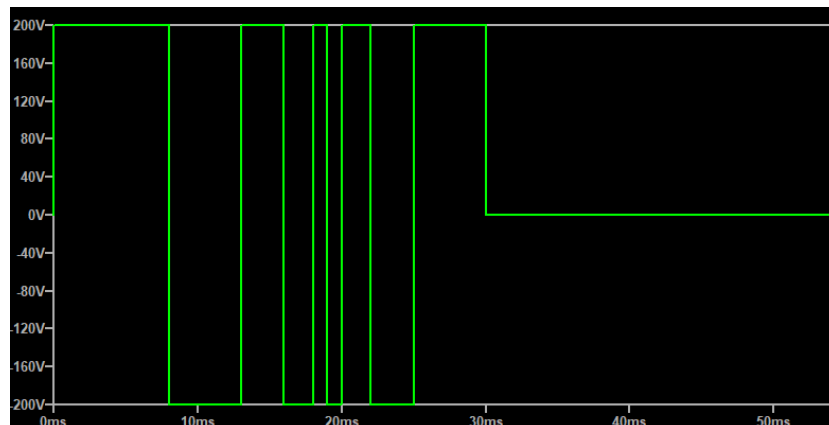
# Number of intervals in the fallback directional H scan
bracket_scan_points=200

```



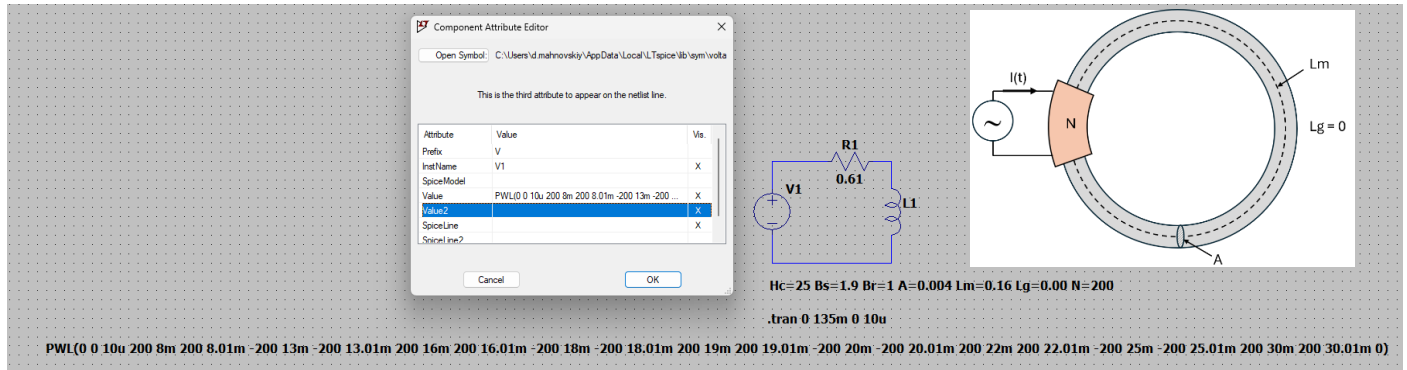


LTspice uses the Chan–Engelhardt model with local magnetic memory (it does not store reversal points). However, we reproduced the same $I(t)$ dependences using the Tellinen–Madelung model with deep magnetic memory. Moreover, even the different slopes of the minor loops in the two models did not affect the current profile. To reveal a possible effect of deep magnetic memory on electrical transients, a more exotic excitation can be attempted. Consider the complex PWM excitation $V(t)$ shown in the figure below. It will generate nested minor loops in both models, but according to different scenarios.



In LTspice, this sequence of rectangular pulses is specified by the following command line in the source Value attribute:

```
PWL(0 0 10u 200 8m 200 8.01m -200 13m -200 13.01m 200 16m 200 16.01m -200 18m -200 18.01m 200 19m 200 19.01m -200 20m -200 20.01m 200 22m 200 22.01m -200 25m -200 25.01m 200 30m 200 30.01m 0)
```



In the Python module V.py, this excitation can be written as follows:

```
def V(t):

    V0 = par["V_test"]

    if t < 0.008:
        return +V0

    elif t < 0.013:
        return -V0

    elif t < 0.016:
        return +V0

    elif t < 0.018:
        return -V0

    elif t < 0.019:
        return +V0

    elif t < 0.020:
        return -V0

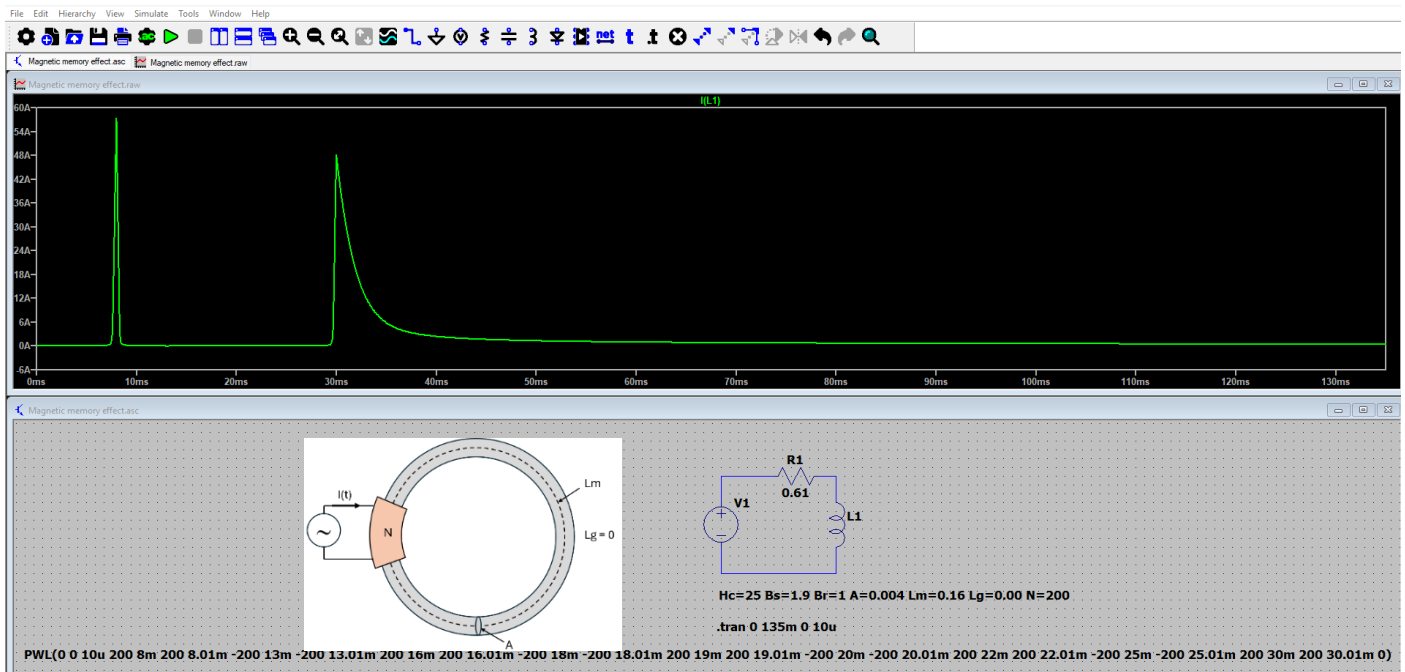
    elif t < 0.022:
        return +V0

    elif t < 0.025:
        return -V0

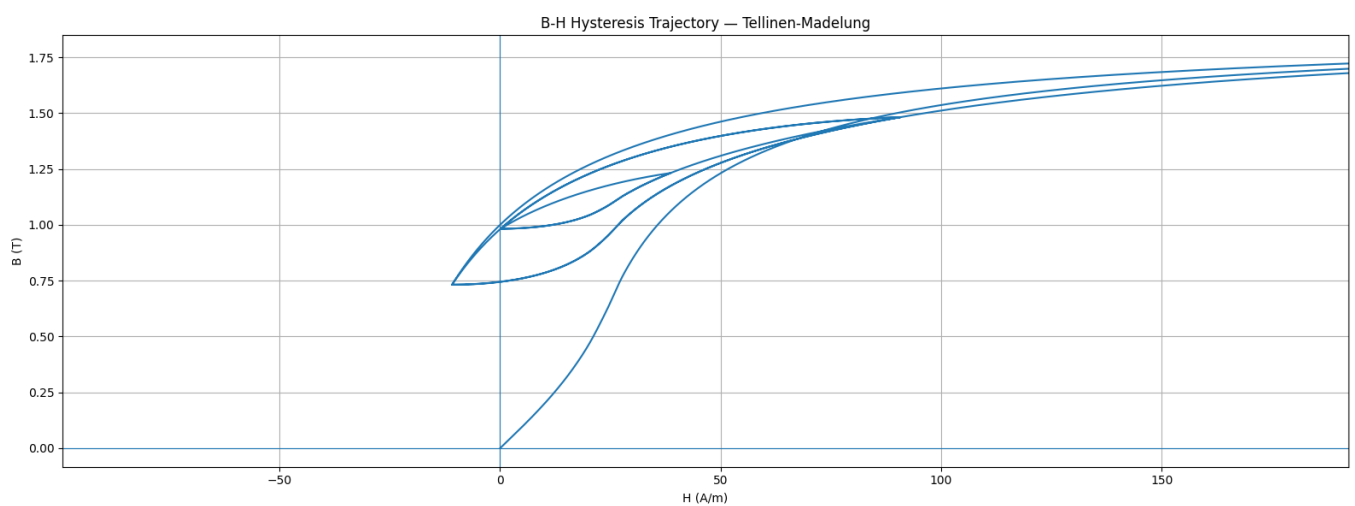
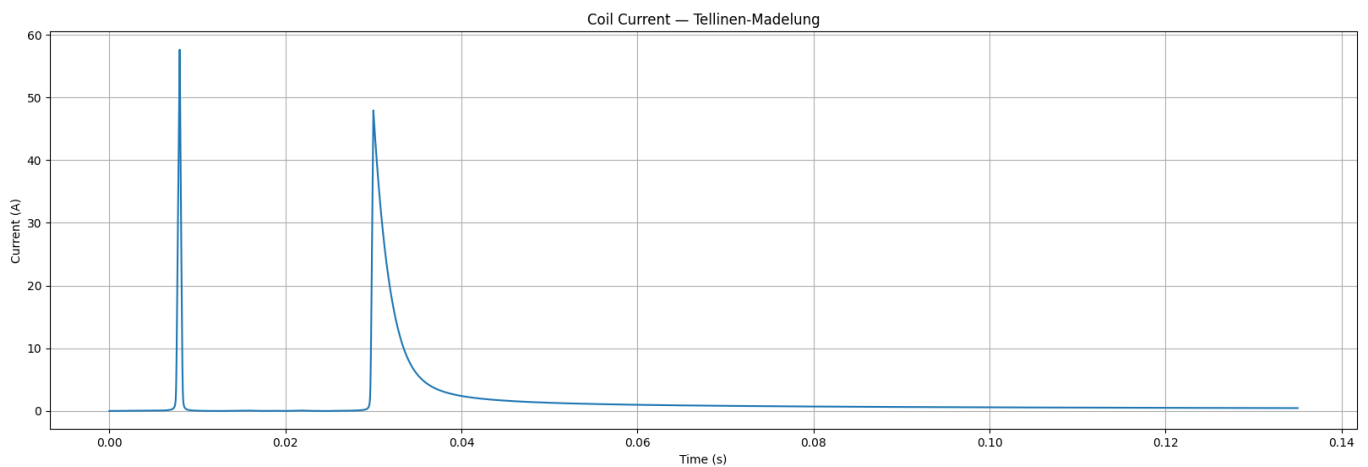
    elif t < 0.030:
        return +V0

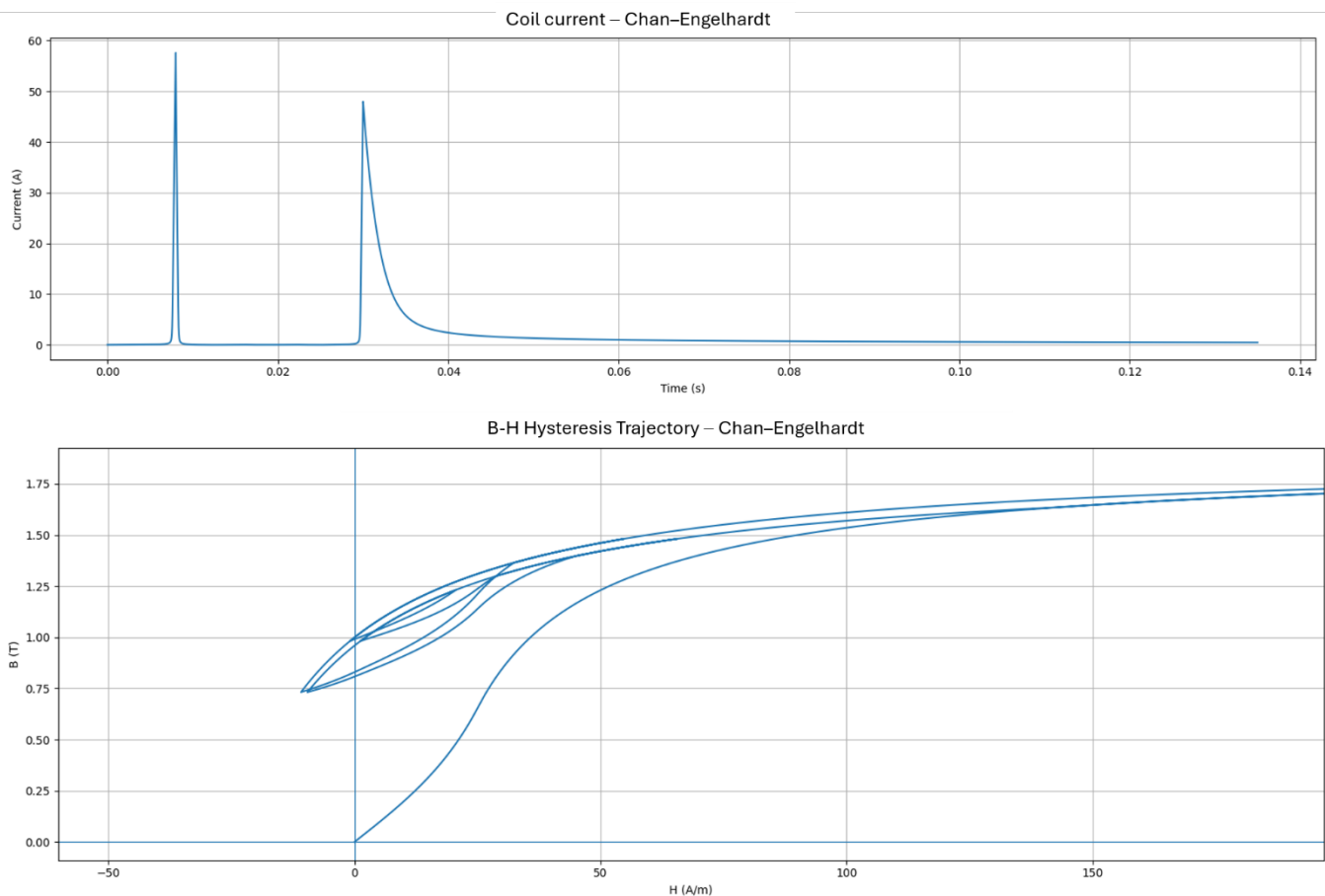
    else:
        return 0.0
```

The current profile $I(t)$ through the inductor, calculated in LTspice using the Chan–Engelhardt model, is shown below.



Now we perform the same simulations using our Python algorithms for both models, with the simulated Chan loop used as the major loop.





The current responses of the two models coincide. Thus, the calculations performed did not reveal any noticeable influence of differences in the implementation of magnetic memory on the transient current of the inductor. Despite the differences in the internal algorithms for storing magnetization history, the Tellinen–Madelung and Chan–Engelhardt models produce coincident electrical responses for the excitation sequences considered.

The comparison shows that, in circuit-simulation problems, the influence of the detailed implementation of magnetic memory on the electrical response may be substantially less important than a correct description of the main nonlinear properties of the magnetic circuit — saturation, coercivity, remanent flux density, and physically plausible behavior of minor loops. Therefore, for many electronics applications there appears to be no need to pursue maximally complex and formally elaborate magnetic-memory models if a simpler model adequately reproduces the observed currents and voltages. This circumstance is particularly important for calculations of the magnetic shaker. It means that the result may be sufficiently robust with respect to a number of simplifying assumptions in magnetic-circuit modeling, including the use of approximate hysteresis models and a limited set of material parameters. In that case, the main attention should be focused on the factors that directly determine the force and transient response of the actuator: the magnetic-circuit geometry, the magnitude and variation of air gaps, saturation of individual sections of the core, the resistance and parameters of the winding, and correct inclusion of the external electrical circuit.

References

- [1] G. Mörée and M. Leijon, “*Review of Hysteresis Models for Magnetic Materials*”, *Energies* 2023, 16, 3908; <https://www.mdpi.com/1996-1073/16/9/3908>
- [2] John H. Chan et al., «*Nonlinear Transformer Model for Circuit Simulation*», *IEEE Transactions on Computer-Aided Design*, vol. 10, No. 4, APRIL 1991.
- [3] Chan–Engelhardt model repository: <https://github.com/DmitriyMakhnovskiy/Chan-Engelhardt-Model-BHloops>
- [4] Tellinen–Madelung model repository: <https://github.com/DmitriyMakhnovskiy/Tellinen-Madelung-Model-BHloops>
- [5] K. Kelp et al., “*Modeling Pulsed Magnetic Core Behavior in LTspice*”, *Electronics* 2025, 14, 2335; <https://www.mdpi.com/2079-9292/14/12/2335>
- [6] M. Th. Engelhardt, “*Asymmetric Minor Hysteresis Loop Model and Circuit Simulator Including the Same*”, U.S. Patent No. 7502723B1, Mar. 10, 2009; <https://patents.google.com/patent/US7502723B1/en>